PYTHON
RECALL

Digital Learning Hub

Python Variables

- Create a new file to start with our refresh >> **variables.py**

- Creating variables syntax:

  varName = value

  print(varName) // to display a variable's value

- Data Types: Integers, floats, Complex **[Numeric Types]**

  Strings, Lists, Tuples, range **[Sequences Types]**

  Dict, sets, frozenset **[Mapping and Set Types]**

  True False [Boolean Type / Logical Type]

  None [NoneType]

- Arithmetic operations :

+	**Addition**
-	**Subtraction**
*	**Multiplication**
**	**Exponentiation**
/	**Division**
//	**Floor Division**
%	**Modulus**

Python Comments ->

Python Comments

To write comments throughout the code in python we have 2 Ways of doing so:

- Single Line comment:

  # This is a Single line comment

- Multiple Line Comment:

```
"""
  This is a Multiple line of comments
  This is the Second line of comment
"""
```

or

```
'''
  This is a Multiple line of comments
  This is the Second line of comment
'''
```

String formatting ->

String Formatting

- Using the **f-String** example:

```
name = "Vanessa"
print(f"Hello {name}")
```

- Using a **String Format** example:

```
message = "Hi {}"
name = "Vanessa"
print(message.format(name))
```

Getting the User Input ->

Getting the User Input

- User input Example:

```
name = input("What is your name?")
age = input("How old are you?")
```

** The input function always stores the value given by the user as a String and assigns it to the variable, which is why mostly when we need the data given for an arithmetic operation we need to convert the data type as the following:

```
age = int(input("How old are you?"))
```

** you can use **float()**, and **str()** too

Lists in Python ->

Lists in Python

- Lists are just collection of data, you can create a list that contains different data types and you cannot access and index out of range

- List example: my_list = [80, 90, '50', 70.45]
  indexes 0 1 2 3
- To select one element from the list you need to refer to it by its index such as:
  print(my_list[2]) >> **it will Display 50**

- To update a value in the list: my_list[2] = **10**
- You can select the last element of any list by using the -1 index as the following: my_list[-1]

- To count how many elements you have in a list : len(my_list)
- To slice the list : my_list[0:2] *we start at index 2 and we stop at index 2 but index 2 is not included*

- To add an element to the list :
  my_list.append(35) -- *it will always add the element at the end of the list*
  my_list.insert(2, 35) -- *it will add the element at the index 2 in the list*

- To delete an element from the list:
  my_list.pop() -- *it will always delete 1 element from the end of the list*
  my_list.remove(35) -- *it will delete the value 35 from the list*

Sets in python ->

Sets in python

- Sets are similar to Lists but they are not ordered and they cannot contain duplications
- Set example : **mySet = {20, 15, 25, 35, 20, 15}**

- To Display all the elements in a set you need to use a loop
- You cannot use indexes with a set

- You can remove an element from the set as the following: **mySet.discard(20)**
- You can empty a set : **mySet.clear()**
- You can add an element to the set : **mySet.add(2)**
- You can add mor than 1 element to a set: **mySet.update([7, 8])**

Tuples in Python ->

Tuples in Python

- Tuples are like lists, ordered but they are unchangeable
- Tuple example : **myTuple = (1, 2, 3, 4, 5)**
- Select 1 element from the Tuple : **myTuple[2]**

Boolean and Operators ->

Boolean and Operators

- A Boolean is a logical data type that takes one of these 2 values : **True** or **False**
- Example of a boolean: **like_coffee** = **True**

- **Comparison Operators:**

  >> 1 == 2 – the result of that comparison will result with **False**
  >> 1 != 2 – the result of that comparison will result with **True**
  >> 1 > 2 – the result of that comparison will result with **False**
  >> 1 < 2 – the result of that comparison will result with **True**
  >> 1 >= 1 – the result of that comparison will result with **True**
  >> 1 <= 2 – the result of that comparison will result with **Ture**

- **Logical Operators:**

  >> 1 > 2 **and** 5 < 7 – The result will be **False**
  >> 1 > 2 **or** 5 < 7 – The result will be **True**
  >> **not(**1 == 1) – The result will be **False**

Control flow conditional statements ->

Control flow conditional statements

- Take this example:

```
x = 1
if x == 1:
    print("X is equal to 1")
elif x == 2:
    print("X is equal to 2")
else:
    print("The value of X is unknown")
```

- You can use the **continue** and **break** statements with you conditions

Control Flow loops statements ->

Control flow loops statements

- Take the following example:

```
my_list = [1, 2, 3, 4, 5]

1) for x in my_list:
        print(x)

2) for x in range(3, 6):
        print(x)

3) i = 0
   while i < 5:
      print(my_list[i])
```

Dictionaries in Python ->

Dictionaries in Python

- A Dictionary is a set of **key – value** pairs as the following:

```
my_dict = {
  'username' : "vanessa",
  'age' : 25
}
```

- Display a certain value from that Dictionary: `print(my_dict.get('username'))`

- Adding a key to the Dictionary : `my_dict['favorite_food'] = "Chicken"`
- Delete a key from the Dictionary : `my_dict.pop('age')`
- Empty a Dictionary: `my_dict.clear()` **or** `del my_dict`

- Copy a Dictionary : `my_dict2 = my_dict.copy()`

- Loop in a Dictionary:

```
for x, y in my_dict.items():
    print(x, y)
```

Functions in Python ->

Functions in Python

- Functions are way to isolate code, and only be able to do certain aspects of code when we want that functionality o be called. Examples:

```
1) def my_function():
       print("Hello world!")

   my_function()

2) def my_function(name):
       print(f"Hello {name}!")

   my_function(name="vanessa")

2) def multiply(a, b):
       return a * b

 result = multiply(3, 10)
```

Imports in Python ->

Imports in Python

- We can create a python file that contains functions we create isolated and import them in our main file where we are typing the code.
- We can also import functions from python modules ready to be used by anyone
- We can import using methods:

```
>> import math
>> from math import pi
>> from math import *
>> import math as mat
```

Object Oriented Programming in Python ->

OOP in Python

- Object Oriented Programming is a programming paradigm based on the concepts of objects which can contain data and code

- OOP benefits:
  - >> Scalability
  - >> Efficiency
  - >> Reusability

- We can define objects by **behavior** or by **state**
- Example:

```
class Cat:
        legs : int = 4
        ears: int = 2
        type : str = "Angora Turkish"
        age : int = 5
        color : str = "White"
cat = Cat() # cat object
cat.legs
cat.ears
```

Small OOP demo Given ->

Small OOP demo Given

We will create a small game where we can create enemies that fight each other

- **Enemies that can fight each other**
- **Different types of enemies (zombie, ogre etc..)**
- **Each enemy have different powers and health points and attack damage**

We will be implementing the four pillars of OOP:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

1. We will create an Enemy Object: ***[we will create is as a class]***

   >> Name / Type of Enemy
   >> Health points
   >> Attack damage

Demo Structure ->

Demo Structure

- Create a file **<<enemy.py>>:**

```
class Enemy: [Bad Practice]
        type_of_enemy : str
        health_points: int = 10
        attack_damage : int = 1
```

- In the **<<main.py>>** file:

```
from Enemy import *

zombie = Enemy()

 zombie.type_of_enemy = 'Zombie'

print(f"{zombie.type_of_enemy } has {zombie.health_points} health points and can
do attack of {zombie.attack_damage }")
```

Good Practice using constructor ->

Good Practice using constructor

- Good practice for the Enemy Class:

```
class Enemy:
    def __init__(self,enemy_type : str, health : int = 10, damage : int = 1):
        self.type_of_enemy: str = enemy_type
        self.health_points: int = health
        self.attack_damage = damage
```

** <<self>> refers to the current class Object

Abstraction in Python ->

Abstraction in Python

- Abstraction means to **hide the implementation** and only **show necessary details** to the user

- We will create more methods in our enemy class:

```
def talk(self):
        print(f'I am a {self.type_of_enemy}. Be prepared to fight!')

def walk_forward(self):
        print(f' {self.type_of_enemy} Moves closer to you.')

def attack(self):
        print(f' {self.type_of_enemy} attacks for {self.attack_damage} damage.')
```

Use these methods in the main.py

Encapsulation in Python ->

Encapsulation in Python

- Encapsulation means bundling of Data

- We will use encapsulation to prevent the user to change the type of the enemy after being initialized. For that we need to change the public attributes into private attributes by adding **double underscore.** Such as:

```
class Enemy:
    def __init__(self,enemy_type : str, health : int = 10, damage : int = 1):
        self.__type_of_enemy: str = enemy_type
        . . .
```

- When turning an attribute from public to private, wee no longer can change it directly, this is why Encapsulation always comes with a **SETTER** to set a value and a **GETTER** to get the value such as:

```
def get_type_of_enemy(self):
        return self.__type_of_enemy

def set_type_of_enemy(self, type_of_enemy):
        self.__type_of_enemy = type_of_enemy
```

Inheritance in Python ->

Inheritance in Python

- Inheritance – Process of acquiring properties from one class to another class. It creates a type of hierarchy between classes (Parent – Child classes)

- Example:

```
class Animal: -- Parent Class
        weight : int
        color : str
        age : int
        animal_type : str

        def eat(self):
                print('Animal Eating')


        def sleep(self):

            print('Animal Sleeping')
```

```
class Dog(Animal):
        ** All Animal Attributes **
        can_shed : bool
        domestic_name : str
        ** All Animal Methods **

        def talk(self):
            print('Bark')
        def eat(self): -- Overriding a method
            print('Chews on Bone')
```

```
dog = Dog()
dog.talk() # Bark
dog.eat() # Chews on Bone
```

Self vs super in Python ->

Self vs Super in Python

- **Self** is used to refer to the current object that is created or being instantiated, while **super** is used to refer to the Parent class
- **Self** is used when there is a need to differentiate between instance variables & parameters with the same name while **super** is used to call the Parent class methods and/or constructors
- Takes a **Person** and **Student** classes example, where the student class inherits from the Person class, and see how their constructors would be created:

```
def __init__(self, name, age): - - > Constructor for the Person Class
        self.name = name
        self.age = age

def __init__(self, name, age, degree): - - > Constructor for the Student Class
        super(). __init__(name = name , age = age)
        self.degree = degree
```

Implementing Inheritance in our demo ->

Implementing Inheritance in our demo

- The **Enemy** will be our **Parent** class and we will create **2 Children** classes **Zombie** and **Ogre**
- Zombie Class:

```
class Zombie(Enemy):
        def __init__(self, health_points, attack_damage):
            super(). __init__(
                        type_of_enemy = 'zombie' ,
                        health_points= health_points ,
                         attack_damage= attack_damage
             )

        def talk(self): # Method Overriding
                        print('Grumbling … ')
        def spread_diseaase(self):
                        print('Zombie is trying to spread infection')

class Ogre(Enemy):
        def __init__(self, health_points, attack_damage):
            super(). __init__(
                        type_of_enemy = 'Ogre' ,
                        health_points= health_points ,
                         attack_damage= attack_damage
             )

        def talk(self): # Method Overriding
                        print('Ogre is slamming Hand all around')
```

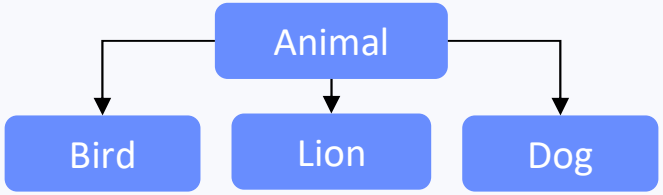
Polymorphism in Python ->

Polymorphism in Python

- **Polymorphism** means to have many forms. It means changing an object at a specific runtime

- Take the Animal and Dog classes as example:

```
zoo = Animal[]

dog = Animal()
dog2 = Dog()
bird = Bird()
lion = Lion()

zoo.append(dog)
zoo.append(dog2)
zoo.append(bird)
zoo.append(lion)

for animal in zoo:
        animal.talk()
```



 # Each have a method

Implementing polymorphism in our demo ->

Implementing polymorphism in our demo

- We will create a new battle function within our **<<main.py>>** file. This function will use our Enemy **talk** and **attack** methods such as the following:

```
def battle(Enemy e):
        e.talk()
        e.attack()
```

- It will be used as the following:

```
zombie = Zombie(10,1)
ogre = Ogre(20,3)

battle(zombie)
battle(ogre)
```

Improvements with our demo ->

Improvements with our demo

- We will give our Enemies a special attack

  >> If the Enemy does not have a special attack, we print "Enemy does not have a special attack"

  >> We will add code in the Enemy (Parent) and both Zombie and Ogre

- In the **Enemy** Class:

```
def special_attack(self):
        print('Enemy has no special attack')
```

- In the **Zombie** Class:

```
def special_attack(self):
        did_special_attack_work = random.random() < 0.50
        if did_special_attack_work :
                self.health_points += 2
                print('Zombie regenerated 2 HP!')
```

- In the **Ogre** Class:

```
def special_attack(self):
        did_special_attack_work = random.random() < 0.20
        if did_special_attack_work :
                self.attack_damage += 4
                print('Ogre attack has increased by 4!')
```

Improvements with our demo ->

Implementing polymorphism in our demo

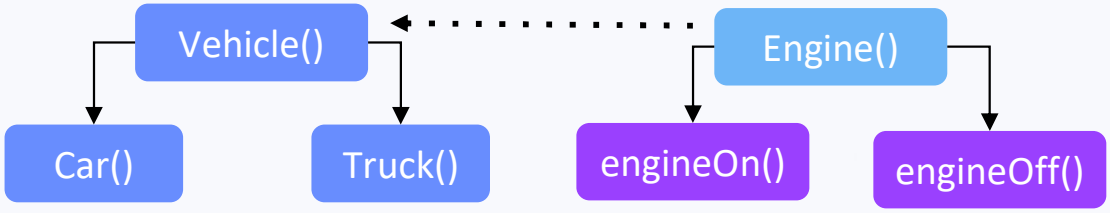
- Update the **battle** function as the following:

```
def battle(e1: Enemy , e2: Enemy):
        e1.talk()
        e2.talk()

        while e1.health_points > 0 and e2.health_points > 0 :
                e1.special_attack()
                e2. special_attack()

                e2.attack()
                e1. health_points  -= e2. attack_damage

                e1.attack()
                e2. health_points  -= e1. attack_damage
        if  e1. health_points > 0 :
                print("Enemy 1 Wins!")
        else:
                print("Enemy 2 Wins!")
```

Composition in python ->

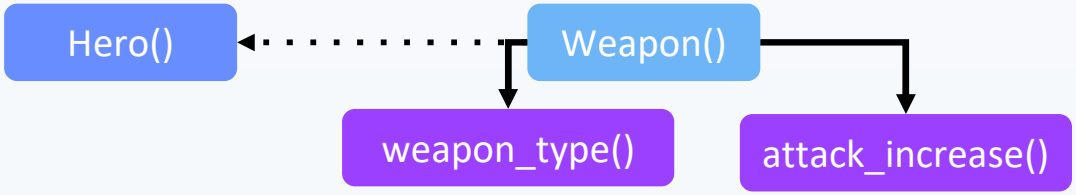
Composition in python

- Composition is a way to create objects made up of other objects. In composition a class contains one or more objects of another class as instance variables, It provides layers of functionality to the object. It is known as **HAS-A** relationship



- To implement Composition in our demo:
  - >> We will create a Hero class
  - >> We will create a new Weapon class

  >> Our Hero will have a **HAS-A (composition)** relationship with our Weapon class



Implementing Composition in our demo ->

Implementing Composition in our demo

- In our **Weapon** class:

```
class Weapon:
        def __init__(self, weapon_type, attack_increase):
                                self.weapon_type = weapon_type
                        self.attack_increase = attack_increase
```

- In our **Hero** class:

```
class Hero:
        def __init__(self, health_points, attack_damage):
                                self.health_points health_points
                        self.attack_damage = attack_damage
                        self.is_weapon_equipped = False
                         self.weapon : Weapon = None

         def equipWeapon(self):
                        if self.weapon is not None and not self.is_weapon_equipped:
                                self.attack_damage +=  self.weapon.attack_increase
                                 self.is_weapon_equipped = True
```

Implementing Composition in our demo ->

Implementing Composition in our demo

- In the **<<main.py>>** file create a **hero_battle function**

```
def hero_battle(hero: Hero , enemy : Enemy ):
        while hearo.health_points > 0 and enemy.health_points > 0 :
                enemy.special_attack()
                enemy.attack()

                hero. health_points  -= enemy. attack_damage

                hero.attack()
                enemy. health_points  -= hero. attack_damage

        if  hero. health_points > 0 :
                print("Hero Wins!")
        else:
                print("Enemy 2 Wins!")
```

- In the **<<main.py>>**:

```
hero = Hero(10,1)
weapon = Weapon('Sword',5)
hero.weapon = weapon
hero.equip_weapon()
hero_battle(hero, zombie)
```

Introduction to FastAPI ->

Digital
Learning
Hub

Develop your digital skills

The mission of the Digital Learning Hub is to help reduce the digital skills gap in Luxembourg.
We do so by offering in-person, affordable courses in various fields of IT and related sectors.

Introduction to FastAPI Web Development with Python

Instructor : Vanessa Al Daham
Course Language: English

Welcome 👋

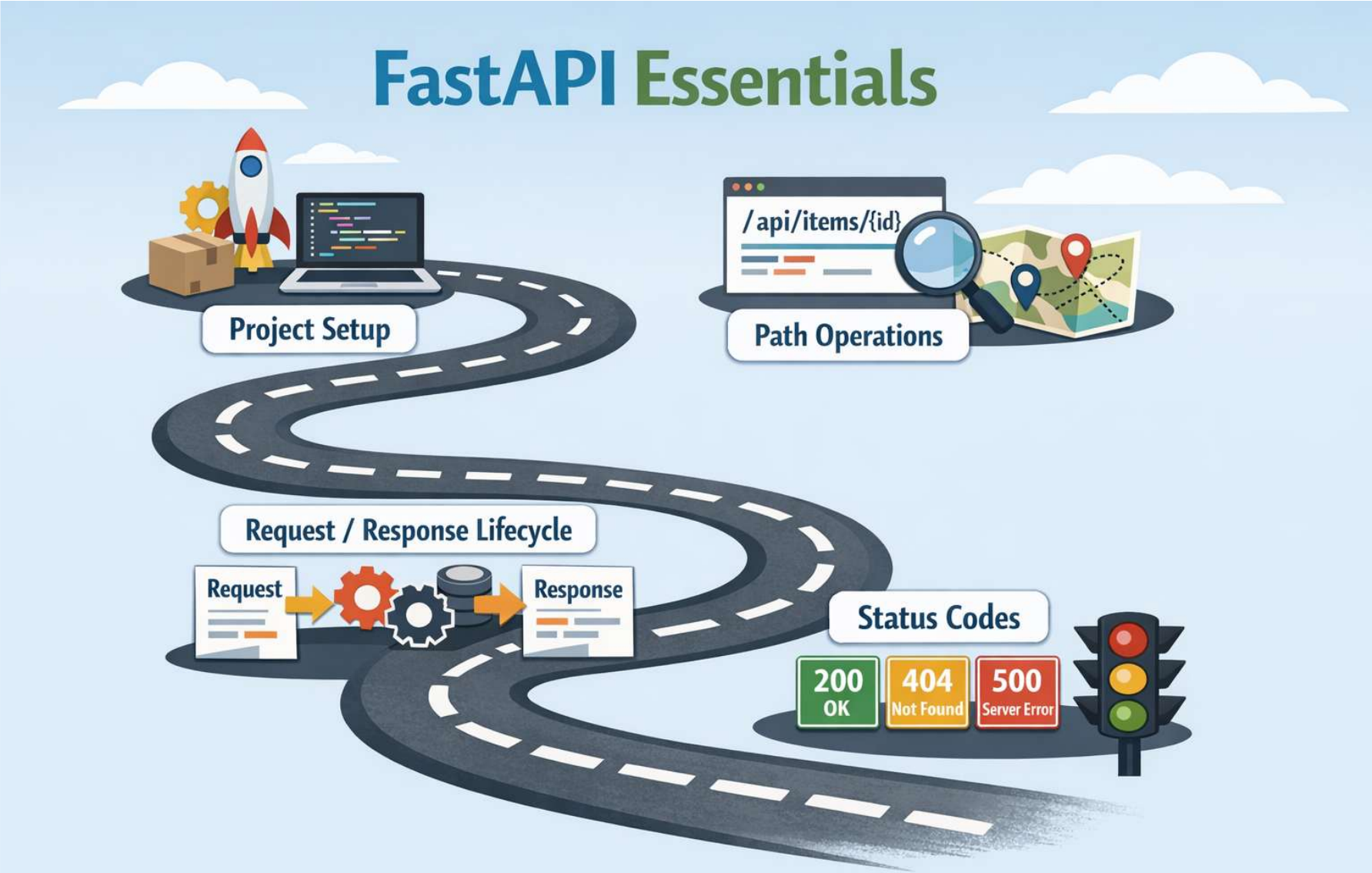
Let’s get to know each other

What’s your name?

Where do you work? Do you have any experience in it?

What are your expectations of this course?

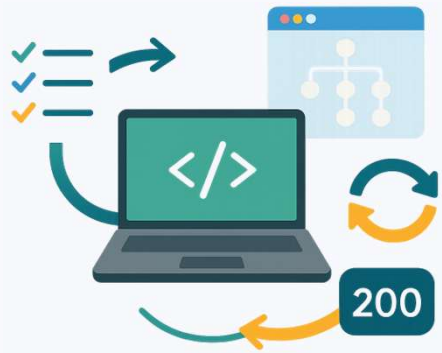
Intermediate Course



Digital Learning Hub

FastAPI Essentials: Project Setup

FastAPI is a modern, high-performance Python web framework for building APIs. It is built on top of Starlette (for web handling) and Pydantic (for data validation), and it automatically generates interactive API documentation using OpenAPI and JSON Schema.



1. **Why FastAPI?**
   - **Extremely fast —** on par with NodeJS and Go thanks to async support via ASGI
   - **Automatic data validation and serialization** via Pydantic models
   - **Auto-generated interactive docs** (Swagger UI and ReDoc) at /docs and /redoc
   - **Built-in support** for type hints, making code easier to read and maintain
   - **First-class async/await support** for non-blocking I/O

FastAPI Installation ->

FastAPI Essentials: FastAPI Installation

2. Installation:

Install FastAPI along with Uvicorn, the ASGI server used to run the application:

```
# Install FastAPI and Uvicorn
pip install fastapi uvicorn

# For full feature support (email validation, etc.)
pip install 'fastapi[all]'
```

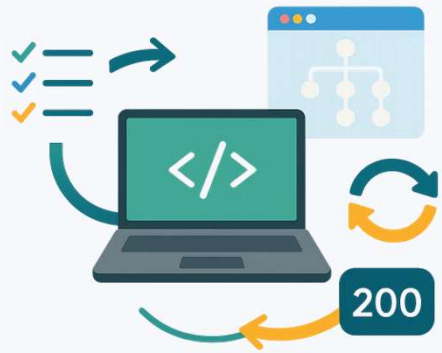
Project Structure ->

Project Setup: Virtual Enviroment Creation

3. Recommended Project Structure:

A clean project layout is essential for maintainability. Here is a recommended structure for a small-to-medium FastAPI project:

```
my_project/
├── app/
|   ├── __init__.py
|   ├── main.py        # App entry point
|   ├── models.py      # Pydantic models
|   ├── routers/       # Route modules
|   |   ├── __init__.py
|   |   ├── users.py
|   |   └── items.py
|   ├── dependencies.py  # Shared dependencies
|   └── database.py     # DB connection
├── tests/
|   └── test_main.py
├── requirements.txt
└── README.md
```



Your First FastAPI App ->

FastAPI Essentials: Your First FastAPI App

Create a folder **<<myproject>>,** open it with VSCode and create a file called **main.py** with the following minimal FastAPI application:

```python
# app/main.py
from fastapi import FastAPI

app = FastAPI(
        title="My API",
        description="A sample FastAPI application",
        version="1.0.0"
)

@app.get('/')
def read_root():
        return {"message": "Hello, FastAPI!"}
```

➢ Run the application with uvicorn, open the terminal and run the following command:

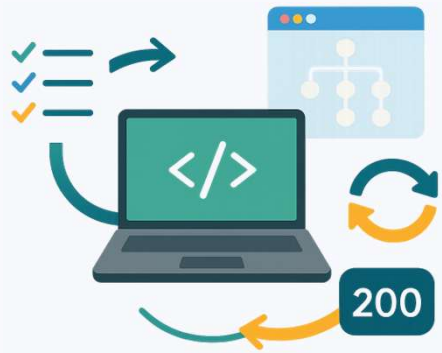
```
uvicorn app.main:app --reload
```

Using Pydantic for Configuration ->

FastAPI Essentials: Using Pydantic for Configuration

FastAPI integrates with Pydantic's BaseSettings to manage environment variables and config cleanly:

```
# app/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
        app_name: str = 'My API'
        debug: bool = False
        database_url: str

        class Config:
                env_file = '.env’

settings = Settings()

# app/main.py
from app.config import settings

@app.get('/info’)
def app_info():
        return {'name': settings.app_name, 'debug': settings.debug}
```



Tasks Time ->

FastAPI Essentials: Your First FastAPI App

Create a folder **<<myproject>>,** open it with VSCode and create a file called **main.py** with the following minimal FastAPI application:

```python
# app/main.py
from fastapi import FastAPI

app = FastAPI(
        title="My API",
        description="A sample FastAPI application",
        version="1.0.0"
)

@app.get('/')
def read_root():
        return {"message": "Hello, FastAPI!"}
```

➢ Run the application with uvicorn, open the terminal and run the following command:

```
uvicorn app.main:app --reload
```

Using Pydantic for Configuration ->

Project Setup: Tasks Time

Task 1: Bootstrap a FastAPI Project
Create a new FastAPI project from scratch.
Install FastAPI and Uvicorn, set up the recommended folder structure, and create a **main.py** with at least three routes: **a root route**, an **/about** route returning app metadata, and **a /health** route returning a status check.
Run with Uvicorn and verify the Swagger docs at /docs.

Task 2: Add Environment-Based Configuration
Extend your project with a **config.py** using Pydantic BaseSettings.
Define settings for **APP_NAME**, **DEBUG**, **DATABASE_URL**, and **API_KEY** read from a **.env file**.
Expose a **/config** route (only when debug=True) that returns the non-sensitive config values. Test that changing the **.env** file without code changes alters the app behavior.

Tasks Time ->

Project Setup: Tasks Time

Task 3: Modular Routing with APIRouter

Refactor your project to use FastAPI's APIRouter.
Create a **routers/** directory with separate files for **users.py** and **items.py**, each defining their own routes with prefixes **(/users, /items)** and tags.

Register both routers in **main.py** using app.include_router().
Ensure all routes still appear in the **/docs** documentation

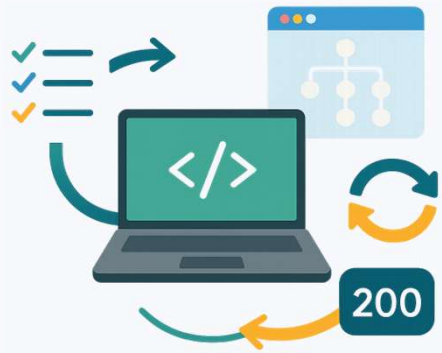
Path Operations ->

FastAPI Essentials: Path Operations

Path operations are the core building blocks of a FastAPI application. They define which HTTP methods respond to which URL paths, what data they accept, and what they return

- **HTTP Methods:**

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/items')            # Read / list resources

@app.post('/items')           # Create a new resource

@app.put('/items/{id}')       # Full update of a resource

@app.patch('/items/{id}')     # Partial update

@app.delete('/items/{id}')    # Delete a resource
```



Path Parameter ->

FastAPI Essentials: Path Parameters

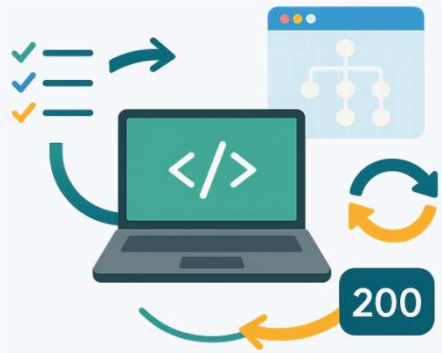
Path parameters are variable parts of the URL path. FastAPI automatically validates and converts them based on type annotations:

```python
from fastapi import FastAPI, Path
from enum import Enum

app = FastAPI()

# Basic path parameter with type coercion
@app.get('/users/{user_id}')
def get_user(user_id: int):
    return {'user_id': user_id, 'type': type(user_id).__name__}
    # GET /users/42 -> {'user_id': 42, 'type': 'int'}

# Path parameter with validation using Path()
@app.get('/products/{product_id}')
def get_product(product_id: int = Path(gt=0, le=9999)):
    return {'product_id': product_id}
```

Path Parameters ->

FastAPI Essentials: Path Parameters

```
# Enum-constrained path parameter
class ModelName(str, Enum):
    alexnet = 'alexnet'
    resnet = 'resnet'
    lenet = 'lenet'

@app.get('/models/{model_name}')
def get_model(model_name: ModelName):
    return {'model': model_name, 'message': f'Using {model_name}'}
```



Query Parameter ->

FastAPI Essentials: Query Parameters

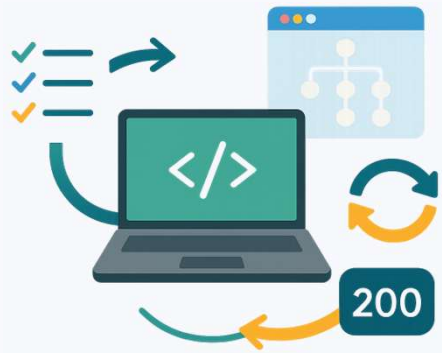
Query parameters appear after the **?** in a URL. FastAPI reads them from the function signature for any parameter not part of the path:

```python
from fastapi import FastAPI, Query

from typing import Optional

app = FastAPI()

# Required query parameter
@app.get('/search')
def search(q: str):
    return {'query': q}
    # GET /search?q=fastapi

# Optional query parameter with default
@app.get('/items')
def list_items(skip: int = 0, limit: int = 10):
    return {'skip': skip, 'limit': limit}
    # GET /items?skip=20&limit=5
```

Query Parameters ->

FastAPI Essentials: Query Parameters

```
# Optional with None default + validation
@app.get('/products')
def list_products(
        category: Optional[str] = None,
        min_price: float = Query(default=0.0, ge=0),
        max_price: float = Query(default=1000.0, le=99999)
):
        return {'category': category, 'price_range': [min_price, max_price]}
```



Request Body with Pydantic Models ->

FastAPI Essentials: Request Body with Pydantic Models

For **POST**, **PUT**, and **PATCH** operations, you define request bodies using Pydantic models. FastAPI automatically **validates input**, **parses JSON**, and **generates schema documentation**:

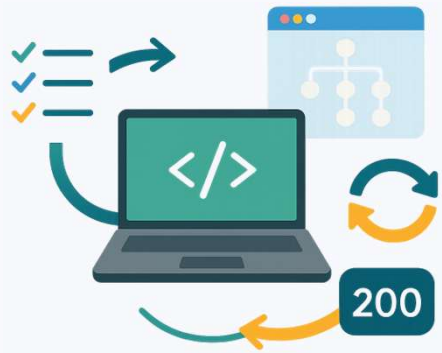
```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import Optional
app = FastAPI()
class Item(BaseModel):
        name: str = Field(..., min_length=1, max_length=100)
        description: Optional[str] = None
        price: float = Field(..., gt=0)
        in_stock: bool = True

        model_config = {
                'json_schema_extra': {
                'examples': [{'name': 'Laptop', 'price': 999.99, 'in_stock': True}]
        }}

@app.post('/items')
def create_item(item: Item):
        return {
                'created': item.model_dump(),
                'name_upper': item.name.upper()
        }
```

Combining Path Query and Body ->

FastAPI Essentials: Combining Path Query and Body

FastAPI intelligently distinguishes between path, query, and body parameters based on their names and annotations:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ItemUpdate(BaseModel):
    name: str
    price: float

# All three types in one endpoint
@app.put('/shops/{shop_id}/items/{item_id}')
def update_item(
    shop_id: int,          # <- path parameter
    item_id: int,          # <- path parameter
    notify: bool = False,  # <- query parameter
    item: ItemUpdate = None # <- request body
):
    result = {'shop': shop_id, 'item_id': item_id}
    if item:
        result['update'] = item.model_dump()
    if notify:
        result['notification'] = 'sent'
    return result
```



Tasks Time ->

FastAPI Essentials : Tasks Time

Task 4: Build a CRUD API for a Book Library
Create a FastAPI application with a complete set of CRUD endpoints for a Book resource.
The Book model should have: id (int), title (str), author (str), year (int, between 1000 and 2100), and available (bool).
Implement GET /books (with skip/limit query params), GET /books/{id}, POST /books, PUT /books/{id}, and DELETE /books/{id}. Use an in-memory list as the data store.

Task 5: Advanced Query Filtering
Add a GET /books/search endpoint that supports filtering by multiple optional query parameters: author (str), min_year and max_year (int), available (bool), and a q parameter for full-text search across title and author.
Use Query() with validation for min/max year ranges. Return a filtered list and a count of results

Tasks Time ->

FastAPI Essentials : Tasks Time

Task 6: Nested Resources and Enum Constraints
Build a nested route structure for a school API: GET /departments/{dept_id}/courses and POST /departments/{dept_id}/courses.
The dept_id should be constrained to a DepartmentEnum (CS, MATH, PHYSICS, HISTORY).
The Course model should include name, code (str, 5-10 chars), credits (int, 1-6), and an optional description. Return 404 if the department is not found

Request/Response Lifecycle ->

FastAPI Essentials: Request/Response Lifecycle

Understanding how FastAPI processes a request end-to-end is key to building reliable APIs. The lifecycle involves middleware, dependency injection, route matching, serialization, and response generation.

- **Lifecycle Overview:** When a client sends a request to your FastAPI application, the following sequence occurs:

Client Request **-> [1]** ASGI Server (Uvicorn) receives the raw HTTP request **->**

[2] Middleware Stack (runs in order: logging, CORS, auth, etc.) **-> [3]** Router matches the URL path andHTTP method

-> [4] Dependencies are resolved (auth, DB session, etc.) **-> [5]** Request body is parsed and validated (Pydantic)

-> [6] Path/Query parameters are validated and coerced **-> [7]** Route handler function executes

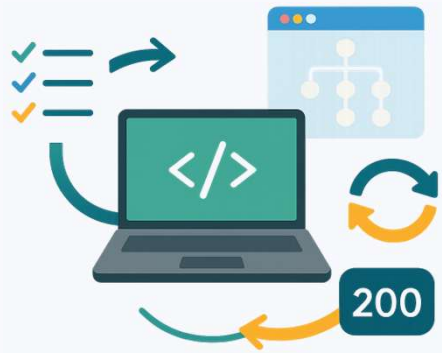
-> [8] Response model validation & serialization **-> [9]** HTTP Response sent to client

Request Object ->

FastAPI Essentials: Request Object

You can access the raw Request object to read headers, cookies, form data, and more:

```
from fastapi import FastAPI, Request

app = FastAPI()

@app.get('/request-info')
async def request_info(request: Request):
        return {
                'method': request.method,
                'url': str(request.url),
                'client_ip': request.client.host,
                'headers': dict(request.headers),
                'query_params': dict(request.query_params),
                'cookies': request.cookies,
        }

# Reading raw body
@app.post('/raw-body')
async def read_body(request: Request):
        body = await request.body()
        return {'raw_bytes': len(body), 'content': body.decode()}
```



Middleware ->

FastAPI Essentials: Middleware

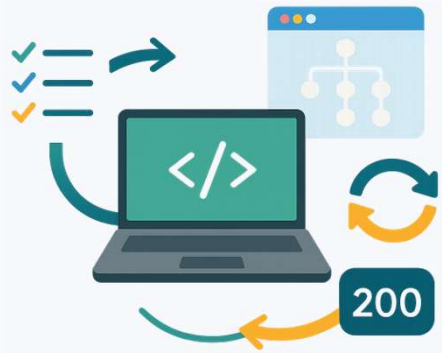
Middleware runs before and after every request. Use it for cross-cutting concerns like logging, timing, authentication headers, and CORS:

```python
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
import time

app = FastAPI()

# CORS Middleware
app.add_middleware(
        CORSMiddleware,
        allow_origins=['https://example.com'],
        allow_methods=['*'],
        allow_headers=['*’],
)
```

Middleware ->

FastAPI Essentials: Middleware

```
# Custom timing middleware

@app.middleware('http’)
async def add_process_time(request: Request, call_next):
        start = time.time()
        response = await call_next(request)   # Execute the route
        duration = time.time() - start
        response.headers['X-Process-Time'] = str(duration)
        return response
```



Dependency Injection ->

FastAPI Essentials: Dependency Injection

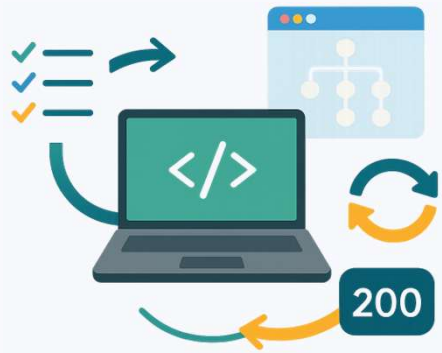
FastAPI's dependency injection system (Depends) allows you to share logic across routes cleanly. Dependencies can themselves have dependencies, forming a tree:

```python
from fastapi import FastAPI, Depends, HTTPException, Header
from typing import Optional

app = FastAPI()

# A simple dependency
def get_db():
    db = {'connection': 'simulated_db'}  # real: SessionLocal()
    try:
        yield db
    finally:
        pass  # real: db.close()
```

Middleware ->

FastAPI Essentials: Dependency Injection

```
# Auth dependency
def verify_token(x_token: str = Header()):
        if x_token != 'secret-token':
                raise HTTPException(status_code=401, detail='Invalid token')
                return x_token

# Both deps used together
@app.get('/secure-data')
def get_data(
        db = Depends(get_db),
         token: str = Depends(verify_token)
):
        return {'db': db, 'token_valid': True}
```



Response Models ->

FastAPI Essentials: Response Models

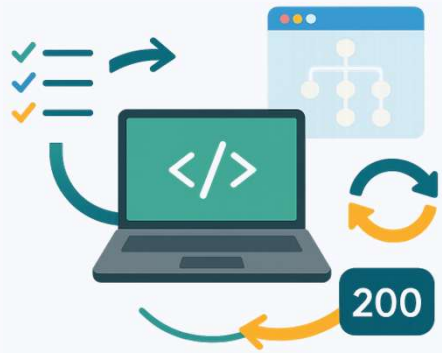
Use the response_model parameter to define what gets returned. FastAPI filters out fields not in the model and validates the output:

```python
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI()

class UserIn(BaseModel):   # Input (includes password)
                    username: str
                    email: str
                    password: str

class UserOut(BaseModel):   # Output (no password!)
                    username: str
                    email: str

fake_db = []
```

Middleware ->

FastAPI Essentials: Response Models

```
# response_model filters the output automatically
@app.post('/users', response_model=UserOut)
def create_user(user: UserIn):
        # Even though we store password, it's NOT returned
        fake_db.append(user.model_dump())
        return user  # FastAPI strips 'password' via response_model

@app.get('/users', response_model=List[UserOut])
def list_users():
        return fake_db
```



Custom Responses ->

FastAPI Essentials: Custom Responses

FastAPI supports returning custom Response objects for cases where you need full control over headers, cookies, or content type:

```python
from fastapi import FastAPI
from fastapi.responses import JSONResponse, HTMLResponse, RedirectResponse
app = FastAPI()

# Custom JSON with extra headers
@app.get('/custom-json')
def custom_json():
    content = {'message': 'Hello with custom headers'}
    headers = {'X-Custom-Header': 'my-value'}
    return JSONResponse(content=content, headers=headers)

# HTML response
@app.get('/page', response_class=HTMLResponse)
def html_page():
    return '<html><body><h1>Hello from FastAPI</h1></body></html>'

# Redirect
@app.get('/old-path')
def redirect():
    return RedirectResponse(url='/new-path', status_code=301)
```

Tasks Time ->

FastAPI Essentials : Tasks Time

Task 7: Implement Request Logging Middleware
Write a custom middleware that logs every incoming request: method, path, query params, client IP, and response status code.
Also add an X-Request-ID header (using uuid4) to each response so clients can trace requests.
Store the last 50 log entries in memory and expose them via a GET /admin/logs endpoint.

Task 8: Dependency Injection with Auth and Pagination
Create a reusable Depends-based pagination dependency that reads skip (default 0) and limit (default 10, max 100) from query params. Also create an auth dependency that reads an Authorization: Bearer <token> header and decodes a mock JWT. Apply both dependencies to a GET /secure/items endpoint that returns a paginated, authenticated list of items

Tasks Time ->

FastAPI Essentials : Tasks Time

Task 9: Response Model Filtering and Transformation
Build a user registration and profile API. The UserCreate model includes: username, email, password, role (admin/user/guest).
Create a UserPublic response model that excludes password and role.
Create a UserAdmin response model that includes all fields.
Implement POST /users (returns UserPublic), GET /users/{id} (returns UserPublic), and GET /admin/users/{id} (returns UserAdmin, requires admin token via Depends).

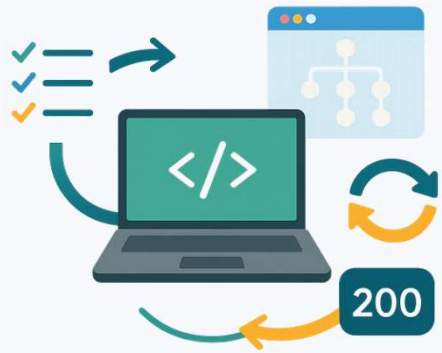
HTTP Status Codes ->

FastAPI Essentials: HTTP Status Codes

HTTP status codes communicate the outcome of a request. FastAPI makes it easy to return the correct code for every situation, either statically via decorators or dynamically inside route handlers

➢ **Status Code Categories:**

Range	Category	Common Examples
1xx	Informational	100 Continue, 101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable



Setting Default Status Codes ->

FastAPI Essentials: Setting Default Status Codes

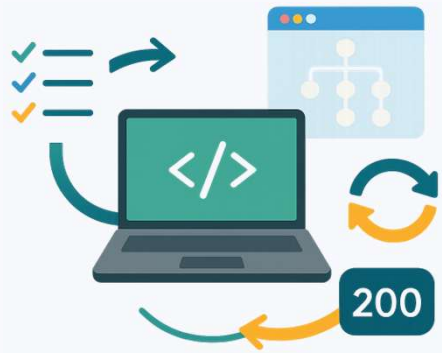
Use the status_code parameter on route decorators to set the default response status code. Import the status module for named constants:

```python
from fastapi import FastAPI, status
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

# 200 OK (default for GET)
@app.get('/items', status_code=status.HTTP_200_OK)
def list_items():
    return []
```

Tasks Time ->

FastAPI Essentials: Setting Default Status Codes

```
# 201 Created (for POST that creates a resource)
@app.post('/items', status_code=status.HTTP_201_CREATED)
def create_item(item: Item):
        return {'id': 1, **item.model_dump()}

# 204 No Content (for DELETE, returns no body)
@app.delete('/items/{item_id}', status_code=status.HTTP_204_NO_CONTENT)
def delete_item(item_id: int):
        return None  # No body sent
```



HTTPException for Error Responses ->

FastAPI Essentials: HTTPException for Error Responses

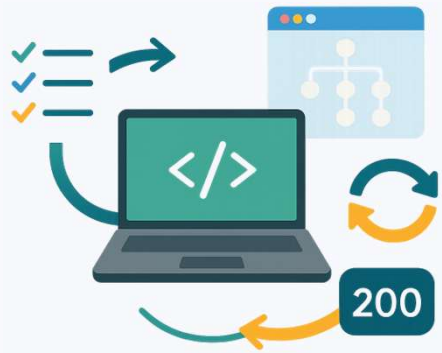
Raise HTTPException to return error status codes from inside your route handlers. FastAPI catches these and returns proper JSON error responses:

```python
from fastapi import FastAPI, HTTPException, status

app = FastAPI()

fake_db = {1: {'name': 'Laptop'}, 2: {'name': 'Phone'}}

@app.get('/items/{item_id}')
def get_item(item_id: int):
    if item_id not in fake_db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f'Item {item_id} not found'
        )
    return fake_db[item_id]
```

HTTPException for Error Responses ->

FastAPI Essentials: HTTP Status Codes

```
# 403 Forbidden
@app.delete('/admin/items/{item_id}')
def admin_delete(item_id: int, is_admin: bool = False):
        if not is_admin:
                raise HTTPException(
                        status_code=status.HTTP_403_FORBIDDEN,
                        detail='Admin access required',
                        headers={'WWW-Authenticate': 'Bearer'}
                )
        return {'deleted': item_id}
```



Validation Errors (422) ->

FastAPI Essentials: Validation Errors (422)

FastAPI automatically returns 422 Unprocessable Entity when request data fails Pydantic validation. The response body contains detailed field-level error information:

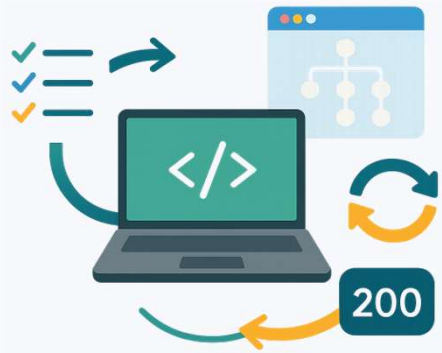
```
# Automatic 422 response example
# If POST /items receives: {'name': '', 'price': -5}
# FastAPI returns:
{
'detail': [
        {
                'loc': ['body', 'name'],
                'msg': 'String should have at least 1 character',
                'type': 'string_too_short'
        },
        {
                'loc': ['body', 'price'],
                'msg': 'Input should be greater than 0',
                'type': 'greater_than'
        }
]}
```

Custom Exception Handlers ->

FastAPI Essentials: Custom Exception Handlers

You can register custom exception handlers to intercept specific exceptions and return formatted responses:

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError

app = FastAPI()

# Custom handler for validation errors
@app.exception_handler(RequestValidationError)
async def validation_error_handler(request: Request, exc: RequestValidationError):
    errors = [{'field': e['loc'][-1], 'message': e['msg']} for e in exc.errors()]
    return JSONResponse(
        status_code=422,
        content={'success': False, 'errors': errors}
    )
```



Custom Exception Handlers ->

FastAPI Essentials: Custom Exception Handlers

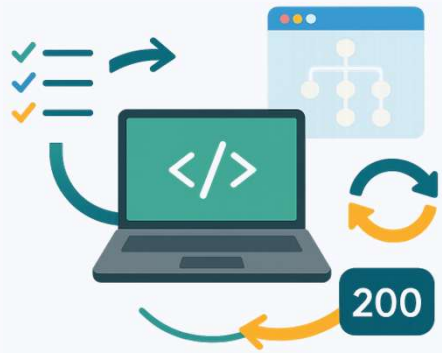
```python
# Custom application exception
class ItemNotFoundError(Exception):
    def __init__(self, item_id: int):
        self.item_id = item_id

@app.exception_handler(ItemNotFoundError)
async def item_not_found_handler(request: Request, exc: ItemNotFoundError):
    return JSONResponse(
        status_code=404,
        content={'error': 'NOT_FOUND', 'item_id': exc.item_id}
    )
```

Dynamic Status Codes with Response Parameter ->

FastAPI Essentials: Dynamic Status Codes with Response Parameter

When you need to return different status codes from the same endpoint (e.g., 200 or 201 depending on whether a resource was found or created), use the Response parameter:

```
from fastapi import FastAPI, Response, status
from pydantic import BaseModel
app = FastAPI()

db = {}
class Item(BaseModel):
        name: str
        price: float

# Returns 200 if item exists, 201 if newly created
@app.put('/items/{item_id}', status_code=200)
def upsert_item(item_id: int, item: Item, response: Response):
        if item_id not in db:
                db[item_id] = item.model_dump()
                response.status_code = status.HTTP_201_CREATED
        else:
                db[item_id].update(item.model_dump())
        return db[item_id]
```



Common Status Code ->

FastAPI Essentials: Common Status Code

Code	Meaning	When to use in FastAPI
200	OK	Default GET; resource found and returned
201	Created	POST creating a new resource successfully
204	No Content	DELETE or action with no response body
400	Bad Request	Invalid syntax; custom business logic errors
401	Unauthorized	Missing or invalid authentication token
403	Forbidden	Authenticated but insufficient permissions
404	Not Found	Resource with given ID does not exist
409	Conflict	Duplicate entry; concurrent modification conflict
422	Unprocessable Entity	Pydantic validation failed (auto by FastAPI)
500	Internal Server Error	Unhandled exceptions; use exception handlers

Tasks Time ->

FastAPI Essentials : Tasks Time

Task 10: Proper Status Codes for a REST API
Review an existing (or newly built) FastAPI CRUD API for a Product resource and ensure every endpoint returns the correct status code: 201 on creation, 200 on retrieval, 204 on deletion (no body), 404 when not found, 409 when trying to create a duplicate product name, and 400 for custom business rule violations (e.g., price cannot be set to 0 for premium products).

Task 11: Custom Exception Handler System
Build a centralized exception handling system. Define custom exception classes: ResourceNotFoundError, DuplicateResourceError, and PermissionDeniedError. Register @app.exception_handler for each, returning consistent JSON: {success: false, error_code: str, message: str}.
Also override the default 422 handler to simplify the validation error format into {success: false, errors: [{field, message}]}.
Test all handlers with dedicated endpoints.

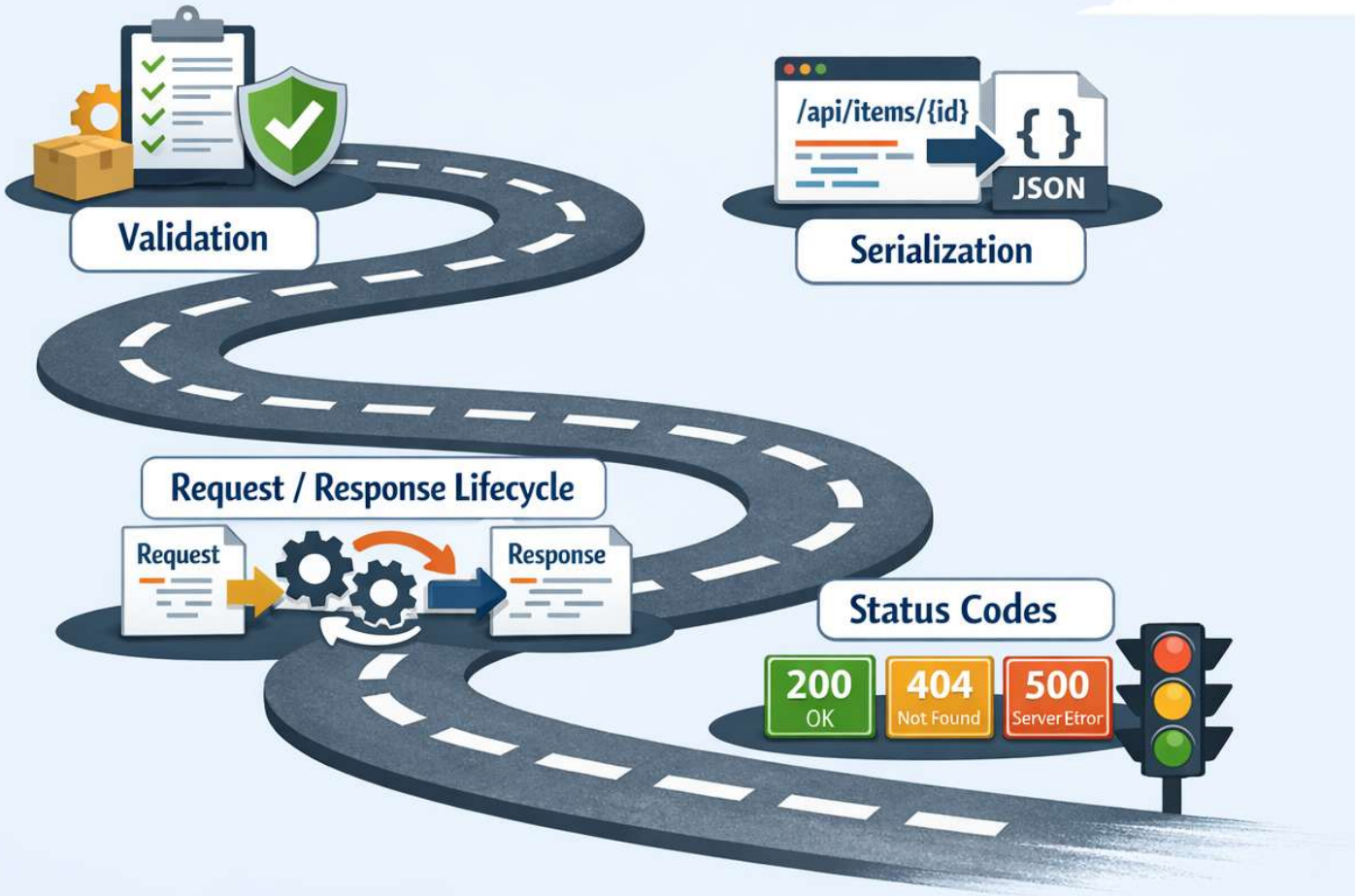
Tasks Time ->

FastAPI Essentials : Tasks Time

Task 12: Upsert Endpoint with Dynamic Status Codes
Implement a PUT /resources/{resource_id} endpoint that acts as an upsert (create or update).
If the resource does not exist, create it and return 201 Created with the new resource.
If it exists, update it and return 200 OK with the updated resource.
Use the Response parameter to set the status code dynamically.
Document both possible responses using the responses parameter in the decorator: responses={200: {'description': 'Updated'}, 201: {'description': 'Created'}}.

Python typing & Pydantic models ->

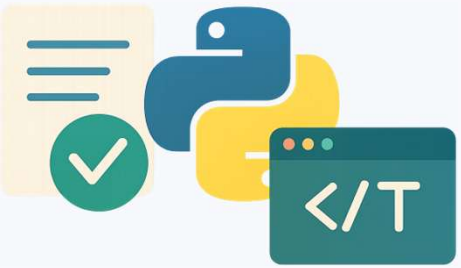
Python Typing & Pydantic Models



Digital Learning Hub

Python typing & Pydantic models : Python Type Hints

Python's type hint system (introduced in PEP 484, Python 3.5+) lets you annotate variables, function parameters, and return values with their expected types. Type hints are not enforced at runtime by default, but they power static analysis tools (mypy, pyright), IDE autocompletion, and — crucially — Pydantic's runtime validation engine



1. Why Type Hints Matter?

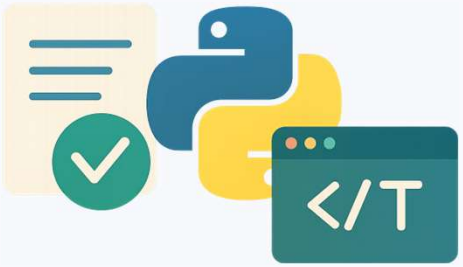
- **Static analysis:** catch bugs before running code with mypy or pyright
- **IDE intelligence:** autocomplete, refactoring, and inline documentation
- **Runtime validation:** Pydantic reads annotations to build validation schemas
- **Self-documenting code:** types serve as always-accurate documentation
- **Improved maintainability:** teams understand data shapes without reading docs

Basic Built-in Types ->

Python typing & Pydantic models : Basic Built-in Types

You can annotate any variable or function parameter with a Python built-in type:

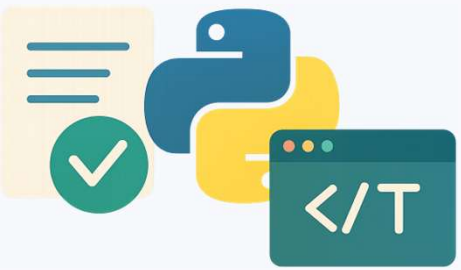
```
# Variables
name: str    = 'Alice'
age:  int    = 30
score: float = 9.5
active: bool = True

# Function signatures
def greet(name: str, times: int = 1) -> str:
        return (f'Hello, {name}! ' * times).strip()

result: str = greet('Bob', 3)
# result -> 'Hello, Bob! Hello, Bob! Hello, Bob!'

# Without hints (still valid Python, but opaque)
def greet_old(name, times=1):
        return (f'Hello, {name}! ' * times).strip()
```



The typing Module ->

Python typing & Pydantic models : The typing Module

For complex types, import from the typing module (Python 3.9+ can use built-ins directly):

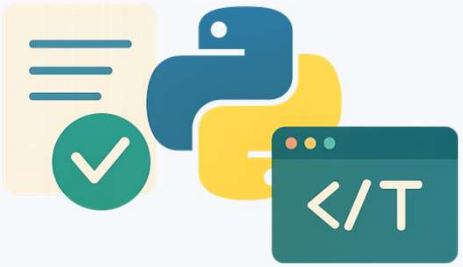


Annotation	Python 3.9+	Description
`Optional[X]`	X | None	Value can be X or None
`Union[X, Y]`	X | Y	Value is X or Y
`List[X]`	list[X]	List whose items are type X
`Dict[K, V]`	dict[K, V]	Dict with key K and value V
`Tuple[X, Y]`	tuple[X, Y]	Fixed-length tuple
`Set[X]`	set[X]	Set of type X
`Callable[[A],R]`	Callable[[A], R]	Callable taking A, returning R
`Any`	Any	Disables type checking
`Literal['a']`	Literal['a']	Exact allowed values
`TypedDict`	TypedDict	Dict with typed keys

Generic Collections ->

Python typing & Pydantic models : Generic Collections

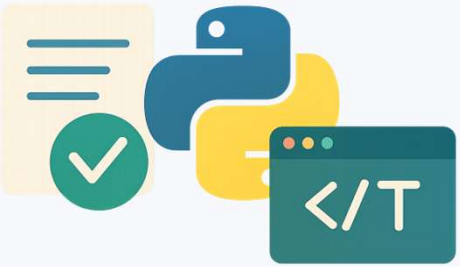
```python
from typing import List, Dict, Tuple, Set, Optional, Union

# Python 3.9+ can use list[], dict[], etc. directly

# List of strings
tags: List[str] = ['python', 'fastapi', 'pydantic']

# Dict mapping string keys to integer values
scores: Dict[str, int] = {'alice': 95, 'bob': 87}

# Optional – value OR None
nickname: Optional[str] = None     # same as Union[str, None]

# Union – one of several types
identifier: Union[int, str] = 42    # or '42'

# Nested generics
matrix: List[List[float]] = [[1.0, 2.0], [3.0, 4.0]]

# Tuple with fixed positions
point: Tuple[float, float] = (3.14, 2.71)

# Modern Python 3.10+ union syntax
value: int | str | None = 'hello'
```



Literal, Final, and TypedDict ->

Python typing & Pydantic models : Literal, Final, and TypedDict

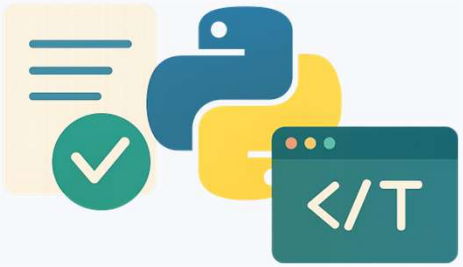
```python
from typing import Literal, Final, TypedDict

# Literal – constrain to specific values
Direction = Literal['north', 'south', 'east', 'west']

def move(direction: Direction, steps: int) -> str:
    return f'Moving {direction} by {steps} steps'

move('north', 3)   # OK
move('up',    3) # mypy error: 'up' is not a valid Direction

# Final – constant that cannot be reassigned
MAX_RETRIES: Final[int] = 3

# TypedDict – dict with known typed keys
class UserRecord(TypedDict):
    id:    int
    name:  str
    email: str

user: UserRecord = {'id': 1, 'name': 'Alice', 'email': 'alice@example.com'}
```



Type Aliases and NewType ->

Python typing & Pydantic models : Type Aliases and NewType

```python
from typing import NewType

# Simple type alias (both are identical at runtime)
UserId  = int
Email   = str
JsonDict = dict[str, object]

def send_email(to: Email, subject: str) -> None: ...

# NewType – creates a distinct type for stronger checking
UserId = NewType('UserId', int)
OrderId = NewType('OrderId', int)

def get_user(user_id: UserId) -> dict: ...

uid = UserId(42)
oid = OrderId(99)

get_user(uid)  # OK
get_user(oid)  # mypy error: OrderId is not UserId
get_user(42)   # mypy error: plain int is not UserId
```

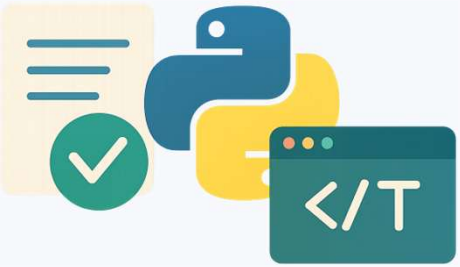


Generics and TypeVar ->

Python typing & Pydantic models : Generics and TypeVar

```python
from typing import TypeVar, Generic, List

T = TypeVar('T')

# Generic function – works with any type
def first_item(items: List[T]) -> T:
    return items[0]

first_item([1, 2, 3])        # returns int
first_item(['a', 'b', 'c'])  # returns str

# Generic class
class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: List[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()
```

```python
int_stack: Stack[int] = Stack()
int_stack.push(42)      # OK
int_stack.push('hello') # mypy error
```



Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 1: Annotate a Data Processing Pipeline

Create a Python module called pipeline.py that processes student records. The module must use type hints throughout. Implement the following typed functions:

- parse_record(raw: dict[str, str]) -> dict[str, int | str | float]: converts raw string values to typed values
- filter_passing(records: list[dict], threshold: float = 60.0) -> list[dict]: returns only passing students
- top_n(records: list[dict], n: int) -> list[tuple[str, float]]: returns top N students as (name, score) tuples
- summarize(records: list[dict]) -> dict[str, float]: returns min, max, average, and count

Run mypy on the module and fix all reported errors. Use Literal, Optional, and Union at least once each.

Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 2: Build a Typed Event System

Implement a generic, type-safe event bus using TypeVar and Generic. Create:

- An EventType TypeVar bound to a base Event dataclass
- A typed EventBus[T] class with subscribe(handler: Callable[[T], None]) and publish(event: T) -> None methods
- Three concrete event types: UserCreated, OrderPlaced, PaymentFailed, each with relevant typed fields
- A dispatcher that routes each event type to the correct bus instance

Verify with mypy that subscribing a UserCreated handler to an OrderPlaced bus raises a type error.

Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 3: Typed Configuration with TypedDict and Literal

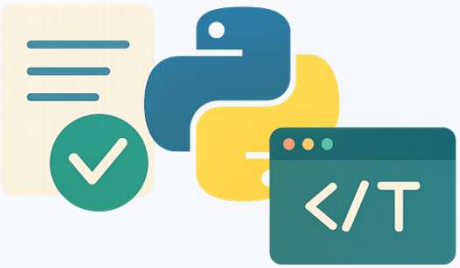
Build a typed configuration loader. Define a ConfigDict using TypedDict with keys: environment (Literal['dev', 'staging', 'prod']), log_level (Literal['DEBUG', 'INFO', 'WARNING', 'ERROR']), max_workers (int), allowed_hosts (list[str]), and feature_flags (dict[str, bool]). Write a load_config(path: str) -> ConfigDict function that reads a JSON file, validates the structure, and raises TypeError with descriptive messages for invalid Literal values. Write a get_feature(config: ConfigDict, flag: str) -> bool helper. Add full mypy annotations and verify with mypy --strict..

Pydantic Models & Validation ->

Python typing & Pydantic models : Pydantic Models & Validation

Pydantic is a data validation library that uses Python type annotations to define schemas and enforce them at runtime. Unlike plain type hints, Pydantic actually validates, coerces, and rejects data — making it the backbone of FastAPI's request/response handling.

Note - Install Pydantic : pip install pydantic

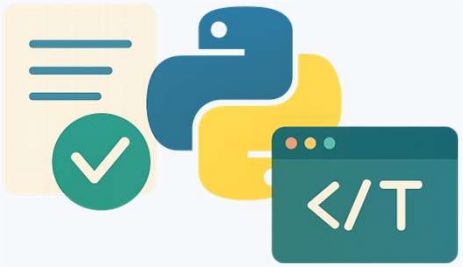


BaseModel Fundamentals ->

Python typing & Pydantic models : BaseModel Fundamentals

Every Pydantic model inherits from BaseModel. Fields are declared as class attributes with type annotations:

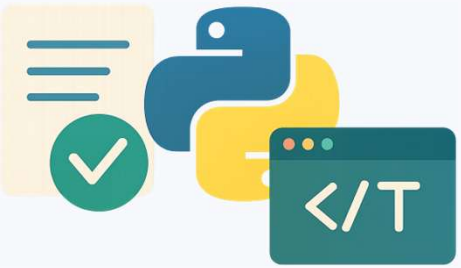
```
from pydantic import BaseModel
from typing import Optional

class User(BaseModel):
        id: int
        username: str
        email:   str
        age:     Optional[int] = None   # optional with default
        active:  bool = True   # required with default

# Instantiation – Pydantic validates automatically
user = User(id=1, username='alice', email='alice@example.com’)
print(user.id)       # 1  (int)
print(user.username)  # 'alice’
print(user.age)      # None
```



BaseModel Fundamentals ->

Python typing & Pydantic models : BaseModel Fundamentals

```
# Auto-coercion: Pydantic converts compatible types
user2 = User(id='42', username='bob', email='bob@example.com')
print(user2.id)      # 42  ← string '42' coerced to int

# Invalid data raises ValidationError
try:
    bad = User(id='not-a-number', username='x', email='x@y.com')
except Exception as e:
    print(e)  # ValidationError with field-level details
```



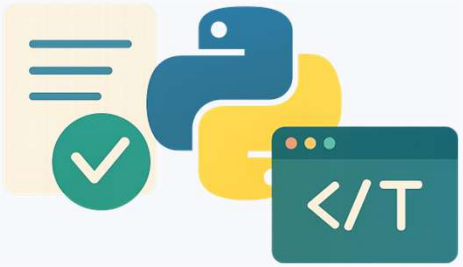
Field() for Advanced Validation ->

Python typing & Pydantic models : Field() for Advanced Validation

Use **Field()** to add constraints, aliases, descriptions, and default factories to any field:

```python
from pydantic import BaseModel, Field
from typing import Optional

class Product(BaseModel):
    # String constraints
    name: str = Field(..., min_length=2, max_length=100)
    sku: str = Field(..., pattern=r'^[A-Z]{2}-\d{4}$')

    # Numeric constraints
    price:  float = Field(...,  gt=0, le=99_999)
    stock:  int   = Field(0,    ge=0)
    discount_pct: float = Field(0.0, ge=0.0, le=1.0)

    # Alias – accept 'product_name' in JSON, expose as 'name' in Python
    display_name: Optional[str] = Field(None, alias='product_name')

    # description appears in OpenAPI/JSON Schema docs
    category: str = Field(..., description='Product category slug')

    # default_factory for mutable defaults
    tags: list[str] = Field(default_factory=list)
```

```python
p = Product(name='Laptop', sku='LT-9001’,
price=999.99, category='electronics’)
print(p.model_dump())
```

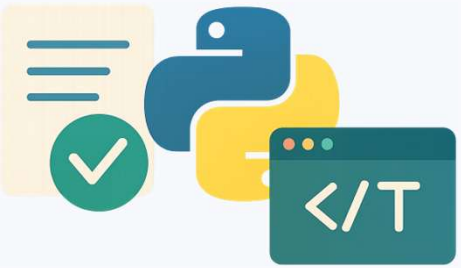


Validators ->

Python typing & Pydantic models : Validators

Pydantic v2 uses @field_validator and @model_validator decorators for custom validation logic:

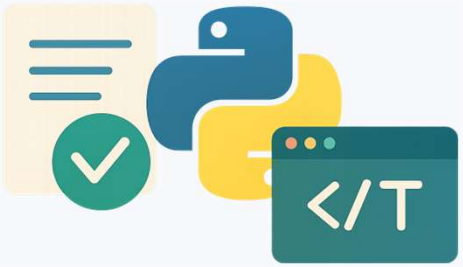
```python
from pydantic import BaseModel, Field, field_validator, model_validator
from typing import Self

class Registration(BaseModel):
    username:       str
    email:          str
    password:       str
    confirm_password: str
    age:            int

    # Field-level validator
    @field_validator('username')
    @classmethod
    def username_alphanumeric(cls, v: str) -> str:
        if not v.isalnum():
            raise ValueError('Username must be alphanumeric')
        return v.lower()   # transform + return
```



Field() for Advanced Validation ->

Python typing & Pydantic models : Validators

```
class Registration(BaseModel):
        . . .
        @field_validator('email')
        @classmethod
        def email_must_have_at(cls, v: str) -> str:
                if '@' not in v:
                        raise ValueError('Invalid email format')
                        return v.lower()
        @field_validator('age')
        @classmethod
        def must_be_adult(cls, v: int) -> int:
                if v < 18:
                        raise ValueError('Must be 18 or older')
                        return v

        # Model-level validator (access multiple fields)
        @model_validator(mode='after')
        def passwords_match(self) -> Self:
                if self.password != self.confirm_password:
                        raise ValueError('Passwords do not match')
                        return self
```

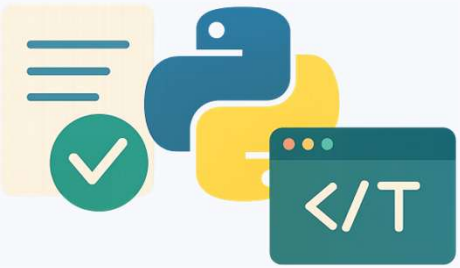


Nested Models ->

Python typing & Pydantic models : Nested Models

Pydantic models can be nested inside each other. Validation cascades automatically through the hierarchy:

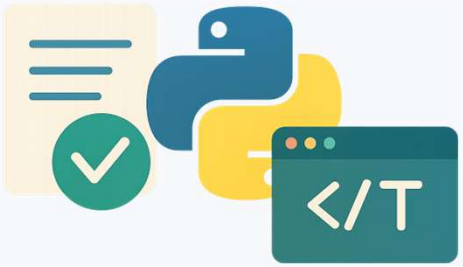
```
from pydantic import BaseModel, Field
from typing import Optional, List

class Address(BaseModel):
        street: str
        city:   str
        country: str = Field(..., min_length=2, max_length=2)  # ISO code
        postcode: Optional[str] = None

class ContactInfo(BaseModel):
        phone:   Optional[str] = None
        website: Optional[str] = None
```



Nested Models ->

Python typing & Pydantic models : Nested Models

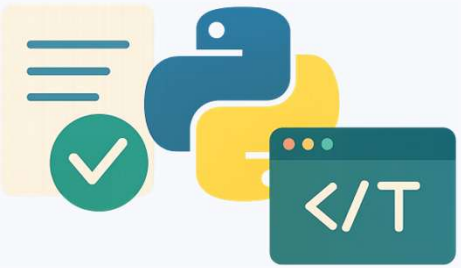
```python
class Company(BaseModel):
    id:       int
    name:     str
    address:  Address          # nested model
    contact:  ContactInfo      # nested model
    branches: List[Address] = [] # list of nested models

# Deep validation from a single dict
data = {
    'id': 1,
    'name': 'Acme Corp',
    'address': {'street': '123 Main St', 'city': 'NYC', 'country': 'US'},
    'contact': {'phone': '+1-555-0100'},
    'branches': [
        {'street': '456 Oak Ave', 'city': 'LA', 'country': 'US'},
    ]
}
company = Company(**data)  # all nested validation fires automatically
```



Discriminated Unions ->

Python typing & Pydantic models : Discriminated Unions

When a field can be one of several model types, use Annotated + discriminated unions for clean, fast dispatch:

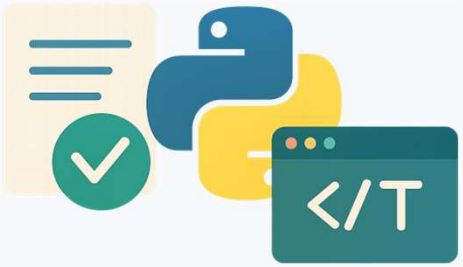


```
from pydantic import BaseModel
from typing import Annotated, Literal, Union

class Cat(BaseModel):
        pet_type: Literal['cat']
        meows_per_day: int

class Dog(BaseModel):
        pet_type: Literal['dog']
        breed: str

class Bird(BaseModel):
        pet_type: Literal['bird']
        can_talk: bool
```

Discriminated Unions ->

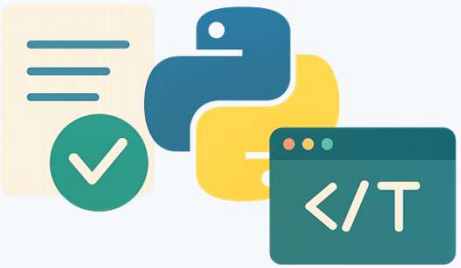
Python typing & Pydantic models : Discriminated Unions

```
# Discriminated union – Pydantic picks the right model
# based on the 'pet_type' field
Pet = Annotated[
   Union[Cat, Dog, Bird],
   Field(discriminator='pet_type’)
]

class Owner(BaseModel):
   name: str
   pet: Pet

o1 = Owner(name='Alice', pet={'pet_type': 'cat', 'meows_per_day': 42})
print(type(o1.pet))   # <class 'Cat'>

o2 = Owner(name='Bob', pet={'pet_type': 'dog', 'breed': 'Labrador'})
print(type(o2.pet))   # <class 'Dog'>
```



model_config and Behaviour Flags ->

Python typing & Pydantic models : model_config and Behaviour Flags



```python
from pydantic import BaseModel, ConfigDict

class StrictUser(BaseModel):
    model_config = ConfigDict(
        # Reject unknown fields instead of ignoring them
        extra='forbid',
        # Do not coerce types – '42' will NOT become 42
        strict=True,
        # Allow population by field name AND alias
        populate_by_name=True,
        # Validate defaults too (not just input data)
        validate_default=True,
        # Freeze model – makes instances immutable (hashable)
        frozen=True,
        # Auto-generate JSON Schema title
        title='Strict User Schema',
    )

    id:   int
    name: str

u = StrictUser(id=1, name='Alice')
# u.id = 2  ← raises ValidationError (frozen=True)
```

Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 4: E-Commerce Order Validation System
Build a complete order validation system with these Pydantic models:

- Address model with street, city, country (ISO 2-char), postcode, and a @field_validator that normalizes country to uppercase
- LineItem model with product_id (int, gt=0), product_name (str), quantity (int, ge=1, le=999), unit_price (float, gt=0), and a computed property total_price
- Order model with id, customer_email, shipping_address (Address), billing_address (Optional[Address], defaults to shipping), items (List[LineItem], min_length=1), status (Literal['pending','paid','shipped','delivered']), and a @model_validator that rejects orders where total exceeds 10,000 without a verified_large_order flag

Test with valid data, invalid emails, empty item lists, and over-limit orders.

Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 5: Recursive Comment Thread Model

Model a nested comment thread where comments can have replies (which are also comments). Create:

- A Comment model with: id (int), author (str), body (str, min_length=1, max_length=2000), created_at (datetime, default to now), likes (int, ge=0), and replies (list[Comment], default empty list)
- A @field_validator on body that strips leading/trailing whitespace and raises ValueError if the result is empty
- A @model_validator that ensures no comment has more than 100 direct replies
- A flatten_thread(comment: Comment) -> list[Comment] helper that r eturns all comments in depth-first order

Construct a 3-level deep thread and call flatten_thread. Verify nested validation fires on deeply nested invalid data.

Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 6: Discriminated Union API Payload Parser
Build a webhook event parser using discriminated unions. Define these event models, each with event_type: Literal[...]:

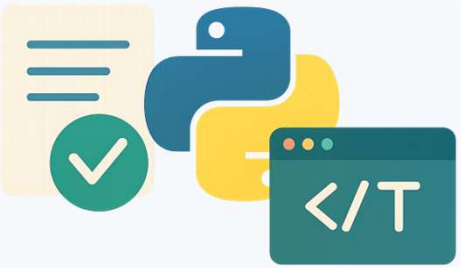
- UserSignedUp: user_id, email, plan (Literal['free','pro','enterprise’])
- PaymentReceived: payment_id, user_id, amount (float, gt=0), currency (Literal['USD','EUR','GBP’])
- SubscriptionCancelled: user_id, reason (Optional[str]), cancelled_at (datetime)
- A WebhookPayload model with: event (discriminated union of all three), webhook_id (str), timestamp (datetime), and signature (str)

Write a process_webhook(raw: dict) -> str function that parses the payload and returns a human-readable summary. Test with payloads for all three event types and one with an unknown event_type.

Serialization ->

Python typing & Pydantic models : Serialization

Serialization converts Python Pydantic objects into JSON-compatible representations (and back). Pydantic v2 provides powerful, configurable serialization with **model_dump()**, **model_dump_json()**, and **model_json_schema().**



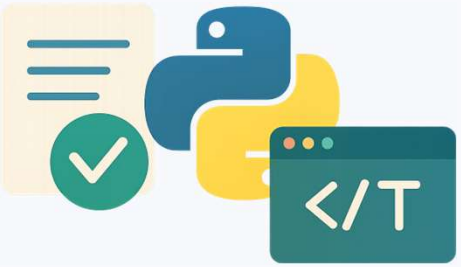
➢ **model_dump() — Python Dict Output:**

```
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime

class Article(BaseModel):
    id:          int
    title:       str
    content:     str
    published_at: Optional[datetime] = None
    internal_tag: str = Field('', exclude=True)  # never serialized

article = Article(id=1, title='FastAPI Guide', content='...', internal_tag='draft')
```

Tasks Time ->

Python typing & Pydantic models: model_dump() — Python Dict Output

```
# Full dict
article.model_dump()
# {'id': 1, 'title': 'FastAPI Guide', 'content': '...', 'published_at': None}
# internal_tag is excluded automatically

# Include only specific fields
article.model_dump(include={'id', 'title'})
# {'id': 1, 'title': 'FastAPI Guide'}

# Exclude specific fields
article.model_dump(exclude={'content'})

# Exclude fields that are None
article.model_dump(exclude_none=True)
# {'id': 1, 'title': 'FastAPI Guide', 'content': '...'}

# Exclude default values
article.model_dump(exclude_defaults=True)
```



Aliases ->

Python typing & Pydantic models : Aliases

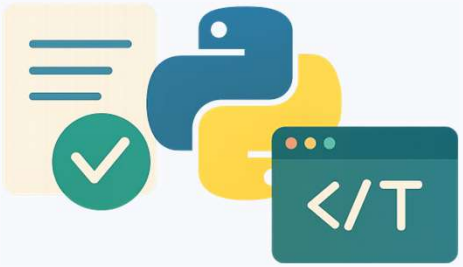
APIs often use camelCase JSON while Python uses snake_case. Pydantic handles this with aliases:

```
from pydantic import BaseModel, Field, AliasGenerator
from pydantic.alias_generators import to_camel

# Manual aliases per field
class UserProfile(BaseModel):
    first_name: str = Field(..., alias='firstName')
    last_name:  str = Field(..., alias='lastName')
    created_at: str = Field(..., alias='createdAt')

# Parse camelCase JSON
u = UserProfile(**{'firstName': 'Alice', 'lastName': 'Smith', 'createdAt': '2024-01-01'})
print(u.first_name)  # 'Alice'
```

Aliases ->

Python typing & Pydantic models: Aliases

```python
# Serialize back to camelCase
u.model_dump(by_alias=True)
# {'firstName': 'Alice', 'lastName': 'Smith', 'createdAt': '2024-01-01'}

# — Auto alias generator (applies globally)
class APIModel(BaseModel):
    model_config = ConfigDict(
        alias_generator=to_camel,    # snake_case -> camelCase automatically
        populate_by_name=True,       # also accept snake_case on input
    )

    user_id:    int
    first_name: str
    is_active:  bool

m = APIModel(user_id=1, first_name='Bob', is_active=True)
m.model_dump(by_alias=True)
# {'userId': 1, 'firstName': 'Bob', 'isActive': True}
```

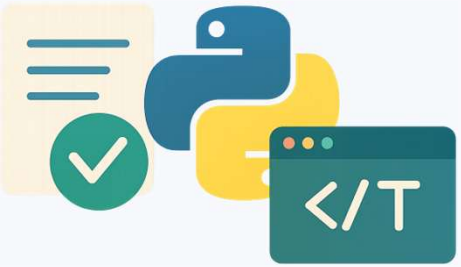


Custom Serializers ->

Python typing & Pydantic models : Custom Serializers

```python
from pydantic import BaseModel, field_serializer
from datetime import datetime

class Event(BaseModel):
    name:       str
    start_time: datetime
    price_cents: int

    # Custom serializer for a field
    @field_serializer('start_time')
    def serialize_dt(self, dt: datetime, _info) -> str:
        return dt.strftime('%Y-%m-%d %H:%M UTC')

    @field_serializer('price_cents')
    def serialize_price(self, cents: int, _info) -> str:
        return f'${cents / 100:.2f}'

event = Event(
    name='Python Conf',
    start_time=datetime(2025, 6, 15, 9, 0),
    price_cents=4999
)
```

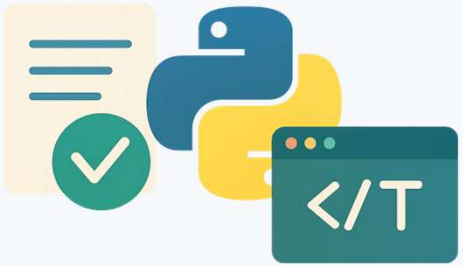
```python
event = Event(
    name='Python Conf',
    start_time=datetime(2025, 6, 15, 9, 0),
    price_cents=4999
)

event.model_dump()
# {
#   'name': 'Python Conf',
#   'start_time': '2025-06-15 09:00 UTC',
#   'price_cents': '$49.99'
# }
```



model_dump_json() — Direct JSON Output->

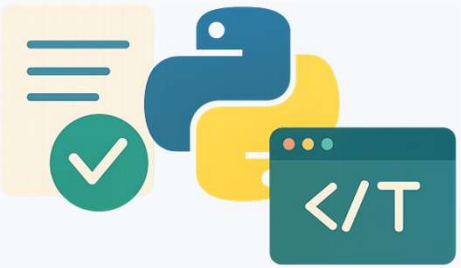
Python typing & Pydantic models : model_dump_json() — Direct JSON Output

```
from pydantic import BaseModel
from datetime import datetime, date
from uuid import UUID, uuid4

class Order(BaseModel):
    order_id:   UUID     = uuid4()
    created_at: datetime = datetime.now()
    due_date:   date
    total:      float

order = Order(due_date=date(2025, 12, 31), total=149.99)
```



model_dump_json() — Direct JSON Output->

Python typing & Pydantic models : model_dump_json() - Direct JSON Output

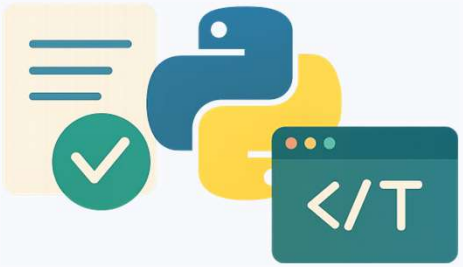


```python
# model_dump() returns a dict (UUID and datetime as Python objects)
print(order.model_dump())
# {'order_id': UUID('...'), 'created_at': datetime(...), ...}

# model_dump_json() returns a JSON STRING (all types serialized)
print(order.model_dump_json(indent=2))
# {
#   "order_id": "3fa85f64-...",
#   "created_at": "2025-01-15T10:30:00",
#   "due_date": "2025-12-31",
#   "total": 149.99
# }

# model_validate_json() – parse from JSON string
json_str = order.model_dump_json()
restored = Order.model_validate_json(json_str)
print(type(restored.order_id))   # <class 'uuid.UUID'>
```

JSON Schema Generation ->

Python typing & Pydantic models : JSON Schema Generation

```python
from pydantic import BaseModel, Field
import json

class BlogPost(BaseModel):
    """A published blog post."""
    title:   str   = Field(..., min_length=5, description='Post title')
    body:    str   = Field(..., min_length=100)
    tags:    list[str] = Field(default_factory=list)
    views:   int   = Field(0, ge=0)

# Generate JSON Schema (used by FastAPI for /docs)
schema = BlogPost.model_json_schema()
print(json.dumps(schema, indent=2))
# {
#   'title': 'BlogPost',
#   'description': 'A published blog post.',
#   'properties': {
#     'title': {'minLength': 5, 'description': 'Post title', ...},
#     ...
#   },
#   'required': ['title', 'body']
# }
```



Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 7: Build a Multi-Format API Serializer
Create a BlogPost Pydantic model with: id (UUID), title (str), body (str), author_name (aliased from 'author'), tags (list[str]), published_at (datetime), reading_time_minutes (int, computed from body length). Implement:

- A @field_serializer for published_at that outputs human-readable format ('January 15, 2025')
- A @field_serializer for tags that joins them as a comma-separated string
- Demonstrate model_dump() with exclude_none and by_alias options
- Demonstrate model_dump_json() and then model_validate_json() round-trip
- Print model_json_schema() and verify all constraints appear correctly

Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 8: camelCase API Bridge

Many frontend frameworks send and expect camelCase JSON. Build a bridge layer:

- Create a CamelModel base class using ConfigDict(alias_generator=to_camel, populate_by_name=True)
- Define UserPreferences, NotificationSettings, and DashboardConfig models that all inherit from CamelModel
- Each model should have at least 4 snake_case fields with various types
- Write a to_api_response(model) -> str function that returns camelCase JSON
- Write a from_api_request(cls, json_str: str) -> Model function that parses camelCase JSON

Verify that both snake_case and camelCase inputs are accepted, but output is always camelCase.

Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 9: Versioned Schema Evolution
Simulate API schema versioning. You have V1 and V2 of a UserProfile model:

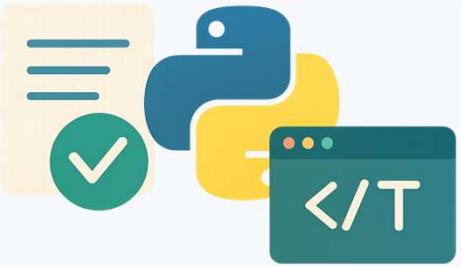
- UserProfileV1: id (int), name (str), email (str), phone (Optional[str])
- UserProfileV2: id (int), first_name (str), last_name (str), email (str), phone (Optional[str]), avatar_url (Optional[str]), metadata (dict[str, str], default empty)
- Write migrate_v1_to_v2(v1: UserProfileV1) -> UserProfileV2 that splits name into first/last
- Write migrate_v2_to_v1(v2: UserProfileV2) -> UserProfileV1 that merges names
- Handle edge cases: single-word names, names with multiple spaces, None phones

Test round-trip migration fidelity with at least 5 different name formats.

Settings Management ->

Python typing & Pydantic models : Settings Management

Pydantic's BaseSettings (from pydantic-settings) extends **BaseModel** to read configuration from environment variables and **.env** files. This is the standard pattern for 12-factor app configuration in Python.
Note: Install: pip install pydantic-settings



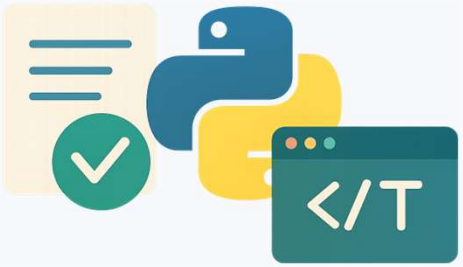
- **Basic BaseSettings:**

```
from pydantic_settings import BaseSettings
. . .
class Settings(BaseSettings):
        app_name:    str  = 'My App'
        debug:       bool = False
        port:        int  = 8000
        secret_key:  str  = Field(..., min_length=32)   # required
        database_url: str = 'sqlite:///./app.db'
        max_workers: int  = 4

        class Config:
                env_file = '.env'        # load from .env file
                env_file_encoding = 'utf-8'

        settings = Settings()
```

Environment-Specific Config Files ->

Python typing & Pydantic models : Environment-Specific Config Files

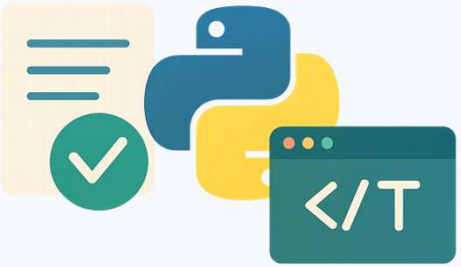
```python
import os
from pydantic_settings import BaseSettings
from pydantic import Field
from typing import Literal

class Settings(BaseSettings):
    environment: Literal['dev', 'staging', 'prod'] = 'dev'
    app_name:    str   = 'My API'
    debug:       bool  = False
    log_level:   str   = 'INFO'
    database_url: str  = Field(...)
    redis_url:   str   = 'redis://localhost:6379'
    cors_origins: list[str] = ['http://localhost:3000']

    model_config = {
        'env_file': ('.env', f".env.{os.getenv('APP_ENV', 'dev')}"),
        'env_file_encoding': 'utf-8',
        'extra': 'ignore',   # silently ignore unknown env vars
    }
```



Nested Settings with Prefixes ->

Python typing & Pydantic models: Nested Settings with Prefixes

```python
from pydantic import BaseModel
from pydantic_settings import BaseSettings

class DatabaseSettings(BaseModel):
    host:     str = 'localhost'
    port:     int = 5432
    name:     str = 'mydb'
    user:     str = 'postgres'
    password: str = ''
    pool_size: int = 10

    @property
    def url(self) -> str:
        return      f'postgresql://{self.user}:{self.password}
                    @{self.host}:{self.port}/{self.name}'
```

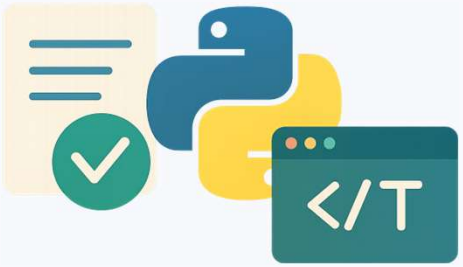


```python
class RedisSettings(BaseModel):
    host:     str = 'localhost'
    port:     int = 6379
    db:       int = 0
    password: str = ''

class AppSettings(BaseSettings):
    app_name: str = 'My API'
    debug:    bool = False

    # Nested models populated from DB__HOST,
    DB__PORT, etc.
    db:    DatabaseSettings = DatabaseSettings()
    redis: RedisSettings    = RedisSettings()

    model_config = {'env_nested_delimiter': '__'}
```

Secrets and Sensitive Values ->

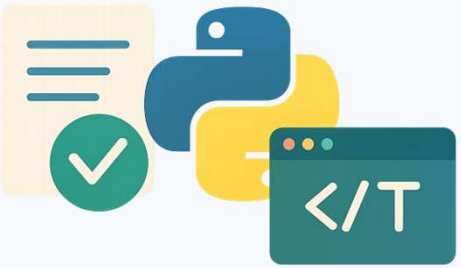
Python typing & Pydantic models: Secrets and Sensitive Values

```python
from pydantic import SecretStr, SecretBytes
from pydantic_settings import BaseSettings

class SecureSettings(BaseSettings):
    # SecretStr hides value in repr() and logs
    secret_key:     SecretStr
    database_password: SecretStr
    api_key:        SecretStr

    model_config = {
        'env_file': '.env',
        # Load individual secrets from files (Docker Swarm / Kubernetes pattern)
        'secrets_dir': '/run/secrets',
    }

settings = SecureSettings()

# SecretStr is opaque in repr() – safe to log
print(settings.secret_key)        # SecretStr('**********')
print(repr(settings.secret_key))   # SecretStr('**********')
# Access the actual value only when needed
actual = settings.secret_key.get_secret_value()
print(actual)   # 'my-real-secret-value'
```



Singleton Pattern with lru_cache->

Python typing & Pydantic models : Singleton Pattern with lru_cache

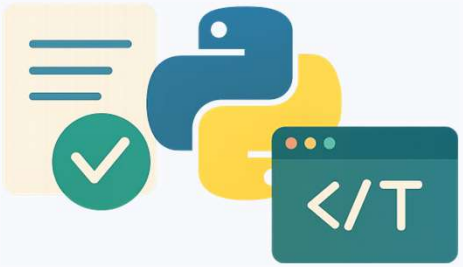


```python
from functools import lru_cache
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    app_name:    str = 'My API'
    debug:       bool = False
    database_url: str = 'sqlite:///./app.db'

    model_config = {'env_file': '.env'}


# lru_cache ensures Settings() is only instantiated once
# (re-reading .env on every call would be wasteful)
@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Singleton Pattern with lru_cache ->

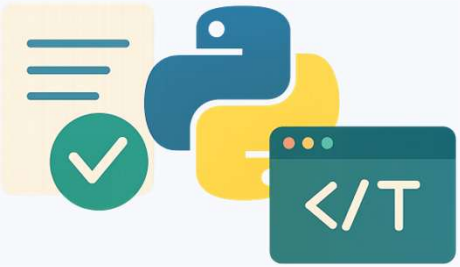
Python typing & Pydantic models : Singleton Pattern with lru_cache

```python
# In FastAPI – use as a dependency
from fastapi import FastAPI, Depends

app = FastAPI()

@app.get('/info')
def app_info(settings: Settings = Depends(get_settings)):
    return {'app': settings.app_name, 'debug': settings.debug}


# In tests – override the cache
from unittest.mock import patch
def test_app_info():
    mock_settings = Settings(app_name='Test App', debug=True)
    with patch('myapp.config.get_settings', return_value=mock_settings):
```



Validators in Settings ->

Python typing & Pydantic models : Validators in Settings

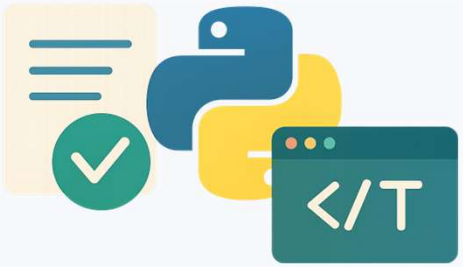


```python
from pydantic import field_validator, model_validator
from pydantic_settings import BaseSettings
from typing import Literal, Self
import re

class Settings(BaseSettings):
    environment:  Literal['dev', 'staging', 'prod'] = 'dev'
    database_url: str
    allowed_hosts: str = 'localhost'  # comma-separated in .env
    max_connections: int = 10

    model_config = {'env_file': '.env'}

    @field_validator('database_url')
    @classmethod
    def validate_db_url(cls, v: str) -> str:
        if not v.startswith(('postgresql://', 'sqlite:///', 'mysql://')):
            raise ValueError('database_url must start with postgresql://, sqlite:///, or mysql://')
        return v
```

Validators in Settings ->

Python typing & Pydantic models : Validators in Settings

```
class Settings(BaseSettings):
        . . .
        @field_validator('allowed_hosts', mode='before')
        @classmethod
        def parse_hosts(cls, v: str) -> list[str]:
                # Convert comma-separated string to list
                return [h.strip() for h in v.split(',') if h.strip()]

        @model_validator(mode='after')
        def validate_prod_settings(self) -> Self:
                if self.environment == 'prod' and self.max_connections < 20:
                raise ValueError('Production requires max_connections >= 20')
        return self
```



Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 10: Multi-Environment Configuration System
Build a production-grade settings system for a web application with these requirements:

- A base Settings class with: APP_NAME, VERSION, ENVIRONMENT (Literal), DEBUG, LOG_LEVEL (Literal), PORT (int), SECRET_KEY (SecretStr, min 32 chars), DATABASE_URL, REDIS_URL (Optional), CORS_ORIGINS (list parsed from comma-separated string via @field_validator), MAX_WORKERS (int), REQUEST_TIMEOUT_SECONDS (int)
- Three .env files: .env (shared defaults), .env.dev (overrides for dev), .env.prod (overrides for prod)
- A @model_validator that enforces: SECRET_KEY cannot be 'changeme' in prod, DEBUG must be False in prod, REDIS_URL must be set in prod
- A get_settings() function cached with lru_cache

Write a settings summary function that prints all settings except SecretStr fields.

Tasks Time ->

Python typing & Pydantic models : Tasks Time

Task 11: Nested Service Configuration with Prefixes
Create a fully nested settings structure for a microservices app using env_nested_delimiter='__':

- DatabaseConfig: host, port, name, user, password (SecretStr), pool_size, max_overflow, echo (bool), with a url @property
- EmailConfig: smtp_host, smtp_port, username, password (SecretStr), from_address, use_tls (bool)
- StorageConfig: provider (Literal['s3','gcs','local']), bucket, region (Optional), access_key (SecretStr), secret_key (SecretStr)
- A root AppConfig(BaseSettings) that contains db (DatabaseConfig), email (EmailConfig), storage (StorageConfig)

Create a .env file that sets DB__HOST, DB__PASSWORD, EMAIL__SMTP_HOST, STORAGE__PROVIDER, etc. Print db.url and all non-secret config values.

Tasks Time ->

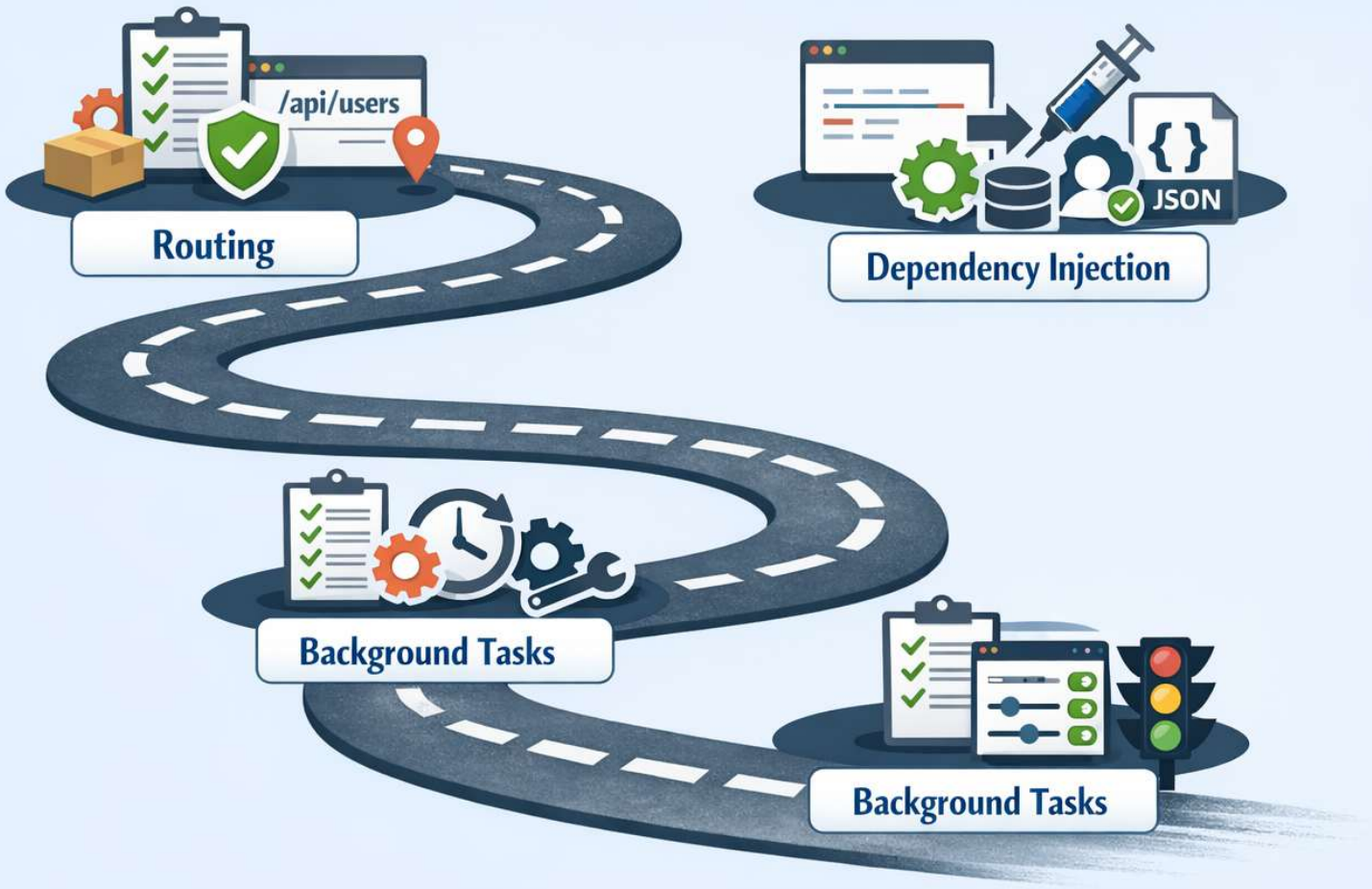
Python typing & Pydantic models : Tasks Time

Task 12: Settings Testing and Overriding
Write a pytest test suite that fully tests your settings system:

- test_default_settings: verify all defaults are set correctly without a .env file (use monkeypatch.delenv to clear env vars)
- test_env_override: use monkeypatch.setenv to set individual env vars and verify they override defaults
- test_prod_validation_errors: verify that invalid prod configs (weak secret, debug=True) raise ValidationError
- test_cors_parsing: verify comma-separated CORS_ORIGINS string is parsed into a list correctly
- test_secret_str_hidden: verify SecretStr fields do not appear in str(settings) or repr(settings)
- test_lru_cache_singleton: verify get_settings() always returns the same instance unless the cache is cleared

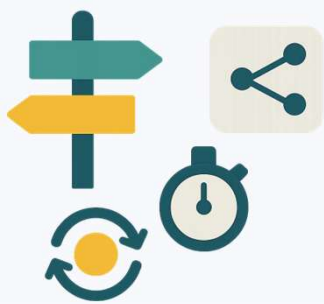
Settings Management ->

Routing & Dependencies



Routing & Dependencies: Routing

Routing is the process of mapping incoming HTTP requests (method + URL path) to specific handler functions in your application. FastAPI's router system lets you organize these mappings into modular, reusable units so your codebase stays clean as it grows.



➢ **Basic Route Registration:**

Every FastAPI route is created by decorating a function with a method decorator on the app (or router) instance. FastAPI automatically handles parameter extraction, validation, serialization, and OpenAPI documentation generation

Example:

```
from fastapi import FastAPI

app = FastAPI()

# GET – retrieve a resource
@app.get('/items/{item_id}')
def get_item(item_id: int): return {'id': item_id}

# POST – create a resource
@app.post('/items', status_code=201)
def create_item(name: str, price: float): return {'name': name, 'price': price}
```

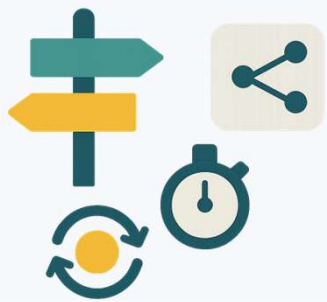
APIRouter ->

Routing & Dependencies: APIRouter

As your application grows, putting all routes in main.py becomes unmanageable. APIRouter lets you define routes in separate modules and register them together in your main app. This is the standard FastAPI project structure pattern.

- **APIRouter vs FastAPI app**

APIRouter is a lightweight routing object with the same decorator API as FastAPI. You define routes on it exactly as you would on the app, then mount it with app.include_router(). This separation keeps your code organized without any performance cost.

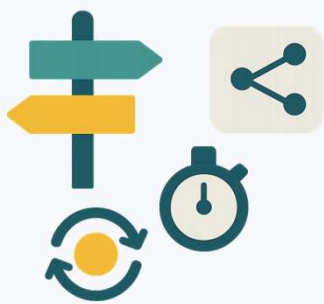


API Router Example ->

Routing & Dependencies: API Router Example

Example:

```python
# app/routers/users.py
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

# Define router with shared prefix and tag for all routes in this file
router = APIRouter(
        prefix='/users',
        tags=['Users'],
        responses={404: {'description': 'User not found'}},
)

class User(BaseModel):
        id: int
        name: str
        email: str

fake_db: dict[int, User] = {}
```

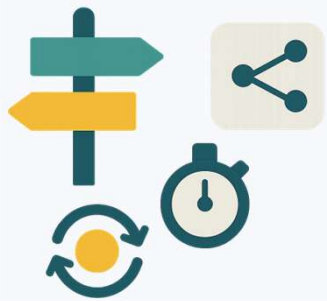


API Router Example ->

Routing & Dependencies: API Router Example

Example Continue:

```
# app/routers/users.py
@router.get('/', response_model=list[User])
def list_users(): return list(fake_db.values())

@router.get('/{user_id}', response_model=User)
def get_user(user_id: int):
        if user_id not in fake_db:
                raise HTTPException(404, detail='User not found')
        return fake_db[user_id]

@router.post('/', response_model=User, status_code=201)
def create_user(user: User):
        fake_db[user.id] = user
        return user
```

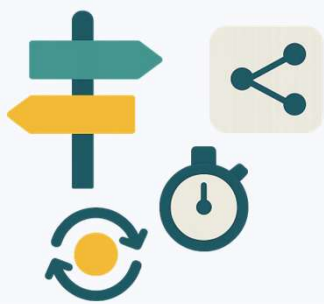


APIRouter Example ->

Routing & Dependencies: API Router Example

Example Continue:

```python
# app/main.py
from fastapi import FastAPI
from app.routers import users, items, orders

app = FastAPI(title='My API', version='1.0.0')

# Register routers
app.include_router(users.router)
app.include_router(items.router)
app.include_router(
        orders.router,
        prefix='/v2', # additional prefix on top of router's own
        tags=['Orders V2'], # override tags
        dependencies=[Depends(verify_auth)],  # apply to ALL routes in
        the router
)
```



Route Parameters in Depth ->

Routing & Dependencies: Route Parameters in Depth

FastAPI distinguishes three parameter sources automatically based on name and type annotation. Understanding this distinction is essential for building correct routes

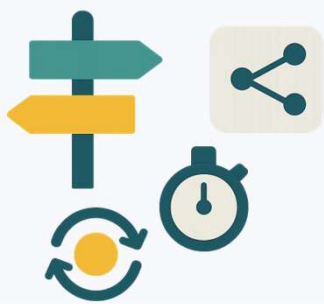
Parameter Type	Location	How FastAPI Detects It	Example
`Path param`	URL path (/items/{id})	Name matches a {} placeholder in path	item_id: int
`Query param`	URL ?key=value	Not in path, not a Pydantic model	skip: int = 0
`Request body`	HTTP body (JSON)	Type is a Pydantic BaseModel subclass	item: Item
`Header param`	HTTP headers	Wrapped in Header()	x_token: str = Header()
`Cookie param`	HTTP cookies	Wrapped in Cookie()	session: str = Cookie()

Route Parameters Example ->

Routing & Dependencies: Route Parameters Example

Example:

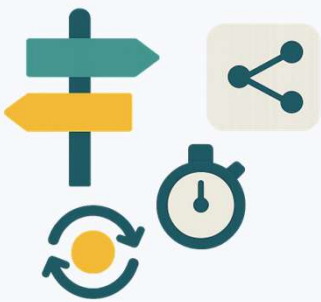
```
from fastapi import FastAPI, Path, Query, Header, Cookie
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class ItemBody(BaseModel):
        name: str
        price: float
```



Route Parameters in Depth ->

Routing & Dependencies: Route Parameters Example

```
@app.put('/shops/{shop_id}/items/{item_id}')
def update_item(
        # Path parameters
        shop_id: int  = Path(..., gt=0, description='The shop ID'),
        item_id: int  = Path(..., gt=0, le=9999),
        # Query parameters
        notify: bool       = False,
        lang:   Optional[str] = Query(None, min_length=2, max_length=5),

        # Request body
        item: ItemBody = None,

        # Header
        x_request_id: Optional[str] = Header(None),
):
        return {
                'shop': shop_id, 'item': item_id,
                'notify': notify, 'lang': lang,
                'body': item.model_dump() if item else None,
                'request_id': x_request_id,
        }
```



Route Ordering and Path Conflicts ->

Routing & Dependencies: Route Ordering and Path Conflicts

FastAPI matches routes in the order they are defined. When two routes could match the same URL, the first one defined wins. This is a common source of bugs with static and dynamic segments at the same level.

Example:

```
from fastapi import FastAPI
app = FastAPI()

#CORRECT ORDER — specific routes BEFORE dynamic ones
@app.get('/users/me')          # matches GET /users/me
def get_current_user(): return {'user': 'current'}

@app.get('/users/{user_id}')    # matches GET /users/42, /users/alice, etc.
def get_user(user_id: str): return {'user_id': user_id}

#WRONG ORDER — /users/me would be caught by {user_id} first!
@app.get('/users/{user_id}')    # 'me' would be user_id='me'
def get_user_wrong(user_id: str): ...

@app.get('/users/me')          # NEVER REACHED
def get_me_wrong(): ...
```

Router Tags, Metadata, and OpenAPI ->

Routing & Dependencies: Router Tags, Metadata, and OpenAPI

Tags group routes in the auto-generated Swagger UI (/docs). You can also add summary, description, and deprecated flags to individual routes for richer API documentation.

Example:

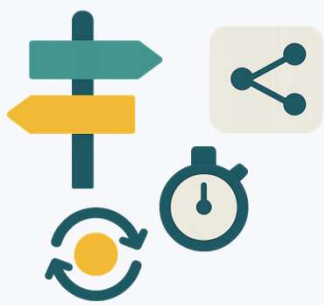
```
from fastapi import APIRouter

router = APIRouter(prefix='/products', tags=['Products'])

@router.get(
            '/{product_id}',
            summary='Get a single product',
            description='Retrieve full details for a product by its unique ID.',
             response_description='The requested product object',
            deprecated=False,
            operation_id='get_product_by_id’,
)
def get_product(product_id: int): return {'id': product_id}
```

Route Ordering and Path Conflicts ->

Routing & Dependencies: Route Ordering and Path Conflicts

Example:

```
# Documenting multiple response codes
@router.post(
        '/',
        status_code=201,
        responses={
                201: {'description': 'Product created successfully'},
                409: {'description': 'Product with this SKU already exists'},
                422: {'description': 'Validation error in request body'},
        }
)
def create_product(name: str, sku: str):
        return {'name': name, 'sku': sku}
```



Nested Routers ->

Routing & Dependencies: Nested Routers

You can mount one APIRouter inside another to build hierarchical URL structures without repeating prefixes everywhere. This is ideal for versioned APIs or resource-scoped sub-resources.

Example:

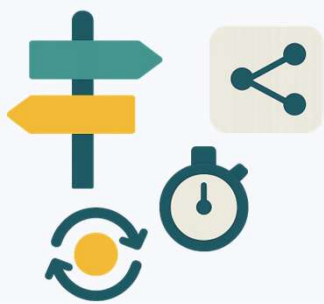
```
from fastapi import FastAPI, APIRouter

# Sub-router: comments inside posts
comments_router = APIRouter()

@comments_router.get('/')
def list_comments(post_id: int):
        return [{'post_id': post_id, 'text': 'Nice post!'}]

@comments_router.post('/', status_code=201)
def add_comment(post_id: int, text: str):
        return {'post_id': post_id, 'text': text}
```

Nested Routers Example ->

Routing & Dependencies: Route Ordering and Path Conflicts

Example:

```
# Parent router: posts
posts_router = APIRouter(prefix='/posts', tags=['Posts'])

@posts_router.get('/')
def list_posts(): return []

# Nest comments under /posts/{post_id}/comments
posts_router.include_router(
                    comments_router,
                    prefix='/{post_id}/comments',
                    tags=['Comments'],
)


# Main app
app = FastAPI()
app.include_router(posts_router)

# Final URL structure:
# GET  /posts
# GET  /posts/{post_id}/comments
# POST /posts/{post_id}/comments
```



Tasks Time ->

Routing & Dependencies : Tasks Time

Task 1: Build a Versioned API with APIRouter

Create a FastAPI application with two API versions (v1 and v2) for a Product resource.

Requirements:

- Create routers/v1/products.py and routers/v2/products.py, each with their own APIRouter
- V1 Product model: id, name, price (float), category (str)
- V2 Product model: id, name, price (float), category (str), tags (list[str]), is_active (bool), created_at (datetime)
- Both versions expose: GET /products, GET /products/{id}, POST /products, DELETE /products/{id}
- Register both routers in main.py with prefixes /api/v1 and /api/v2 and appropriate tags
- Add a GET /api/version route on the main app that lists available API versions

Verify both versions are independently accessible and documented in /docs under separate tag groups.

Tasks Time ->

Routing & Dependencies : Tasks Time

Task 2: Nested Resource Router — Blog Engine
Build a blog engine with a three-level nested routing structure:

- GET/POST /posts — list and create blog posts
- GET/PUT/DELETE /posts/{post_id} — single post operations
- GET/POST /posts/{post_id}/comments — list and add comments
- GET/DELETE /posts/{post_id}/comments/{comment_id} — single comment
- GET/POST /posts/{post_id}/comments/{comment_id}/replies — nested replies

Each model (Post, Comment, Reply) must have at minimum: id (int), author (str), body (str, min 1 char), created_at (datetime). Use separate router files for posts, comments, and replies. Include proper 404 handling at each level with descriptive error messages that include the parent resource ID

Tasks Time ->

Routing & Dependencies : Tasks Time

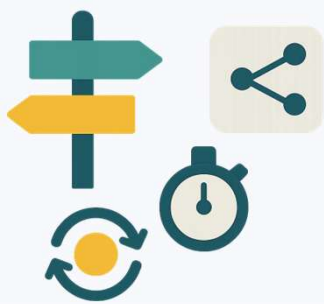
Task 3: Route Metadata and OpenAPI Documentation
Build a fully documented Order Management API where every route has complete OpenAPI metadata:

- Each endpoint must have: summary, description (multi-line), response_description, and tags
- Each endpoint must document all possible response codes with descriptions in the responses={} dict
- Add deprecated=True to at least two old endpoints and create their replacements
- Use operation_id on every route to produce clean, predictable client SDK method names
- Add example values to all Pydantic models using model_config json_schema_extra

Run uvicorn and verify that /docs shows rich documentation for every single endpoint, with no 'No description' placeholders anywhere. Export the OpenAPI JSON from /openapi.json and verify all operation IDs are unique.

Dependencies & Dependency Injection ->

Routing & Dependencies: Dependencies & Dependency Injection



➢ **What is Dependency Injection?**

Dependency Injection (DI) is a design pattern where a component declares what it needs (its dependencies), and a framework provides those dependencies automatically. In FastAPI, the Depends() function is the DI mechanism: you declare what a route needs, and FastAPI resolves, caches, and injects it at request time — with full async support, automatic cleanup, and testability.

➢ **Why Dependency Injection?**

Without DI, you end up with repeated boilerplate in every route: opening DB connections, verifying tokens, reading headers, paginating results. DI solves this elegantly:

- **DRY code** — write auth/pagination/DB logic once, use it in hundreds of routes
- **Testability** — swap real dependencies for test doubles during testing
- **Composability** — dependencies can themselves have dependencies (trees of deps)
- **Lifecycle management** — dependencies can acquire and release resources cleanly
- **Automatic documentation** — FastAPI includes dependency parameters in OpenAPI

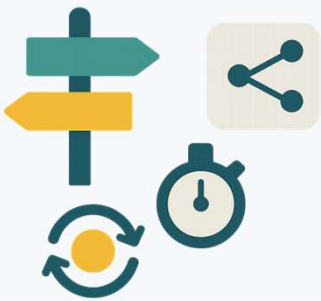
Your First Dependency ->

Routing & Dependencies: Your First Dependency

Any callable (function, class, or async function) can be a dependency. FastAPI reads its type annotations to resolve sub-dependencies recursively.

Example:

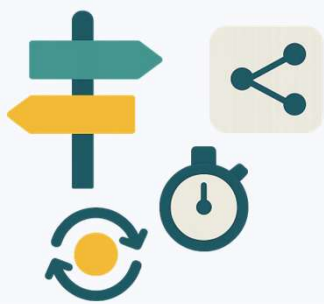
```
from fastapi import FastAPI, Depends
from typing import Optional

app = FastAPI()

# A simple function dependency
def common_pagination(skip: int = 0, limit: int = 10) -> dict: return {'skip': skip, 'limit': limit}

# Use it in a route with Depends()
@app.get('/items')
def list_items(pagination: dict = Depends(common_pagination)):
        return {'pagination': pagination, 'items': []}

# Use the same dep in another route — zero duplication
@app.get('/users')
def list_users(pagination: dict = Depends(common_pagination)):
        return {'pagination': pagination, 'users': []}
```



Your First Dependency Example ->

Routing & Dependencies: Your First Dependency Example

Example:



```
# Class-based dependency (more powerful)
class Pagination:
        def __init__(self, skip: int = 0, limit: int = Query(10, le=100)):
                self.skip  = skip
                self.limit = limit

@app.get('/orders')
def list_orders(p: Pagination = Depends(Pagination)):
        # p.skip and p.limit are ready to use
        return {'skip': p.skip, 'limit': p.limit}

# Shorthand for class deps — omit Depends() argument
@app.get('/products')
def list_products(p: Pagination = Depends()):
        return {'skip': p.skip}
```

Dependencies with yield ->

Routing & Dependencies: Dependencies with yield — Resource Lifecycle

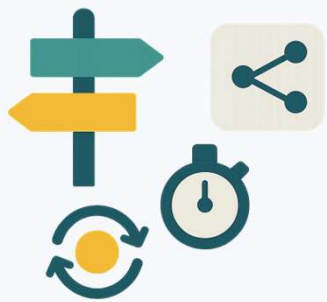
Use yield in a dependency to run cleanup code after the response is sent. This is the standard pattern for database sessions, file handles, and any resource that needs explicit release.

Request arrives

-> Dependency runs up to yield [acquire resource (open DB session)]

-> Yield -> Route handler executes [Uses the yielded resource] -> Response sent to client

-> Cide after yield executes [cleanup – close DB session , release resource]

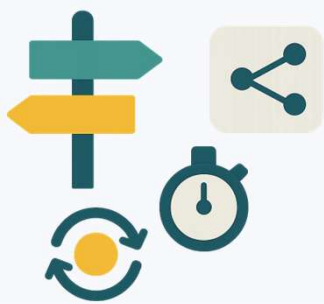


Dependencies with yield Example ->

Routing & Dependencies: Dependencies with yield Example

Example:

```python
from fastapi import FastAPI, Depends
from typing import Generator

app = FastAPI()

# Simulated DB session
class DBSession:
    def __init__(self):
        self.queries: list[str] = []
        print('[DB] Session opened')

    def query(self, sql: str):
        self.queries.append(sql)
        return f'Result of: {sql}'

    def close(self):
        print(f'[DB] Session closed. Ran {len(self.queries)} queries')
```

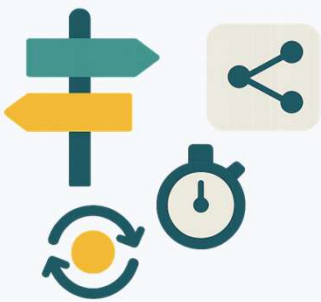


Dependencies with yield Example ->

Routing & Dependencies: Dependencies with yield Example

Example:

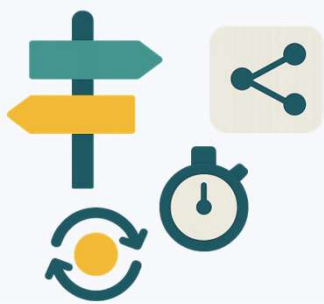
```
def get_db() -> Generator[DBSession, None, None]:
        db = DBSession()
        try:
                yield db        # route handler receives this
        finally:
                db.close()      #  always runs, even if handler raises


@app.get('/items')
def list_items(db: DBSession = Depends(get_db)):
        result = db.query('SELECT * FROM items')
        return {'data': result}

# Each request gets its OWN db session.
# Session is guaranteed closed after the response.
```



Dependency Trees ->

Routing & Dependencies: Dependency Trees

Dependencies can declare their own dependencies. FastAPI builds a tree and resolves them in the correct order. Each dependency is only instantiated once per request, even if used by multiple dependants.



Example:

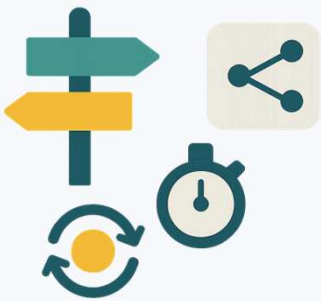
```
from fastapi import FastAPI, Depends, Header, HTTPException
from typing import Optional
app = FastAPI()

# Level 1: lowest-level dependency
def get_db():  # same as before
    db = {'connection': 'open'}
    try:
        yield db
    finally:
        db['connection'] = 'closed'

# Level 2: depends on get_db
def get_token(authorization: Optional[str] = Header(None)) -> str:
    if not authorization or not authorization.startswith('Bearer '):
        raise HTTPException(401, 'Missing token')
    return authorization.removeprefix('Bearer ').strip()
```

Dependency Trees Example ->

Routing & Dependencies: Dependency Trees Example

Example:

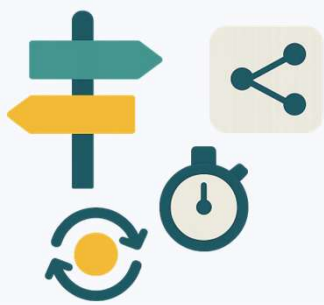
```
# Level 3: depends on both get_db AND get_token
def get_current_user(
        token: str      = Depends(get_token),
        db:    dict     = Depends(get_db),
) -> dict:
        # In reality: decode JWT, look up user in DB
        return {'user_id': 42, 'token': token, 'db_status': db['connection']}


# Route uses only the top-level dep
@app.get('/profile')
def get_profile(user: dict = Depends(get_current_user)):
        return user

# FastAPI resolves the full tree automatically:
# get_profile -> get_current_user -> get_token (reads header)
#             -> get_db    (opens session)
```



Global and Router-Level Dependencies ->

Routing & Dependencies: Global & Router - Level Dependencies

You can apply a dependency to every route in the app, every route in a router, or a single route. This is how you apply authentication, rate limiting, and CORS checks globally.



Example:

```
from fastapi import FastAPI, Depends, APIRouter, Header, HTTPException

# Shared auth dependency
def verify_api_key(x_api_key: str = Header(...)):
        if x_api_key != 'secret-key':
                raise HTTPException(403, 'Invalid API key')

# Option 1: Apply to whole app
app = FastAPI(dependencies=[Depends(verify_api_key)])
# Every single route in the app requires x-api-key header

# Option 2: Apply to a specific router
admin_router = APIRouter(
        prefix='/admin',
        tags=['Admin'],
        dependencies=[Depends(verify_api_key)],  # all routes protected
)
```

Global & Router Example ->

Routing & Dependencies: Global & Router Example

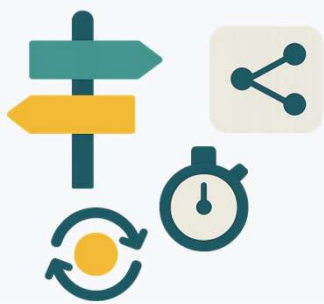
Example:

```
@admin_router.get('/stats')
def admin_stats():  # no need to declare the dep here
        return {'total_users': 1000}


# Option 3: Per-route (most common)
public_router = APIRouter(prefix='/public', tags=['Public'])

@public_router.get('/health')
def health():  # no auth needed
        return {'status': 'ok'}

@public_router.get('/secure', dependencies=[Depends(verify_api_key)])
def secure():  # this one needs auth
        return {'secret': 'data'}
```

Sub-dependencies and Caching ->

Routing & Dependencies: Sub-dependencies and Caching

By default, FastAPI caches a dependency result within a single request. If the same dependency is used multiple times in a dependency tree, it is only called once. You can opt out of caching with **use_cache=False**.



Example:

```
from fastapi import FastAPI, Depends
import uuid
app = FastAPI()

# This dependency generates a request ID
def get_request_id() -> str:
        rid = str(uuid.uuid4())
        print(f'Generating request ID: {rid}')
        return rid


def dep_a(rid: str = Depends(get_request_id)) -> str:
        return f'A got rid={rid}’

def dep_b(rid: str = Depends(get_request_id)) -> str:
        return f'B got rid={rid}'
```

Sub-dependencies and Caching ->

Routing & Dependencies: Sub-dependencies and Caching

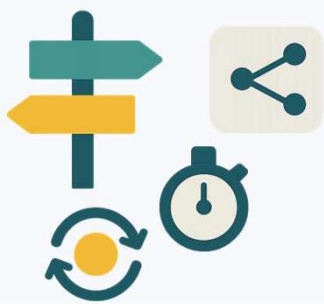
Example:

```
@app.get('/test-cache')
def test_cache(a: str = Depends(dep_a), b: str = Depends(dep_b)):
        # get_request_id is called ONCE even though both dep_a and dep_b use it
        # a and b receive the SAME request ID
        return {'a': a, 'b': b}


# Disable caching
def get_timestamp() -> float:
        import time; return time.time()

@app.get('/no-cache')
def no_cache(
        t1: float = Depends(get_timestamp, use_cache=False),
        t2: float = Depends(get_timestamp, use_cache=False),
):
        # get_timestamp called TWICE — t1 and t2 may differ
        return {'t1': t1, 't2': t2, 'same': t1 == t2}
```

Testing with Dependency Overrides ->

Routing & Dependencies: Testing with Dependency Overrides

FastAPI's app.dependency_overrides dict lets you replace real dependencies with test doubles. This means you can test routes without a real database, real auth server, or real external services



Example:

```
# main.py
from fastapi import FastAPI, Depends

app = FastAPI()

def get_db():
        return {'type': 'real_postgres', 'host': 'prod.db.com'}

def get_current_user(db = Depends(get_db)):
        return {'id': 1, 'name': 'Real User', 'db': db}

@app.get('/me’)
def read_me(user = Depends(get_current_user)):
        return user
```

Testing with Dependency Overrides ->

Routing & Dependencies: Testing with Dependency Overrides

Example:

```
# tests/test_main.py
from fastapi.testclient import TestClient
from main import app, get_db, get_current_user

client = TestClient(app)

def test_read_me_with_override():
    # Override get_db with a fake
    def fake_db():  return {'type': 'sqlite', 'host': 'memory'}
    def fake_user(): return {'id': 99, 'name': 'Test User'}

    app.dependency_overrides[get_db]  = fake_db
    app.dependency_overrides[get_current_user] = fake_user

    try:
        resp = client.get('/me')
        assert resp.status_code == 200
        assert resp.json()['name'] == 'Test User'
    finally:
        app.dependency_overrides.clear()  # always clean up
```

Tasks Time ->

Routing & Dependencies : Tasks Time

Task 4: Layered Authentication System
Build a three-tier authentication dependency chain:

- get_token(authorization: str = Header(...)): extracts and validates Bearer token format
- get_current_user(token = Depends(get_token)): decodes the token (use a mock dict {token: user_dict}), raises 401 if token unknown
- require_admin(user = Depends(get_current_user)): raises 403 if user role != 'admin'
- Apply get_current_user to a /profile endpoint, require_admin to /admin/stats
- Create a /public/info endpoint with NO auth dependency

Write at least 5 pytest tests covering: valid user, valid admin, missing token, wrong token, non-admin trying admin route. Use dependency_overrides for all tests

Tasks Time ->

Routing & Dependencies : Tasks Time

Task 5: Database Session Dependency with Connection Pool
Simulate a connection pool with proper lifecycle management:

- Create a ConnectionPool class with acquire() and release(conn) methods and a max_connections limit
- Write a get_db() dependency using yield that acquires a connection and always releases it (even on error)
- Write a get_transaction() dependency that wraps get_db() and adds commit() / rollback() logic — commit on success, rollback on exception
- Create three routes that use get_transaction(): POST /users (create), PUT /users/{id} (update), DELETE /users/{id}
- Create a GET /pool/status route (no transaction needed) that returns pool metrics

Simulate a route that raises an exception mid-handler and verify the rollback fires. Log all acquire/release/commit/rollback events to confirm lifecycle correctness.

Tasks Time ->

Routing & Dependencies : Tasks Time

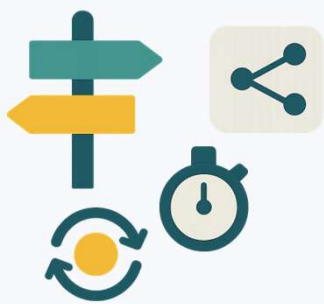
Task 6: Composable Filter Dependencies
Build a reusable, composable set of query filter dependencies for a Product listing API:

- PaginationDep: skip (int, ge=0, default 0), limit (int, ge=1, le=100, default 20)
- SortDep: sort_by (Literal['name','price','created_at'], default 'created_at'), order (Literal['asc','desc'], default 'desc')
- SearchDep: q (Optional[str], min_length 2 when provided), category (Optional[str])
- PriceFilterDep: min_price (float, ge=0, default 0), max_price (Optional[float, le=1_000_000])
- A ProductFilters class that depends on ALL four deps and exposes a .apply(products) method

Apply ProductFilters to GET /products. Test that each filter works independently and all work together. Add a GET /products/export endpoint that uses SortDep and SearchDep but NOT PaginationDep (returns all matching results).

Background Tasks ->

Routing & Dependencies: Background Tasks

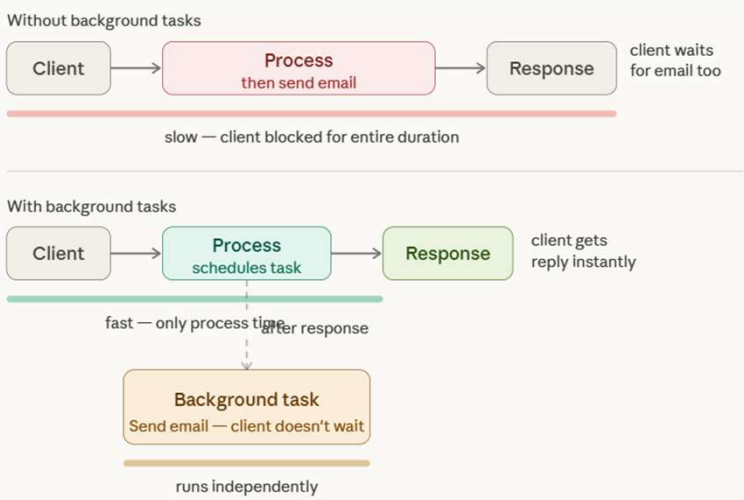


➢ **What are Background Tasks?**

Background tasks are functions that run AFTER a response has already been sent to the client. They are ideal for work that the client does not need to wait for: sending emails, writing logs, updating caches, triggering webhooks, or processing uploaded files. FastAPI's BackgroundTasks is lightweight and runs in the same process, not a separate worker

➢ **How Background Tasks Work**

The flow with background tasks is fundamentally different from normal request handling:



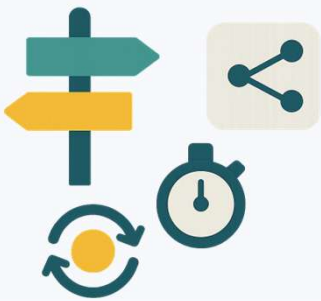
Basic Background Task ->

Routing & Dependencies: Basic Background Task

Inject BackgroundTasks into your route function. Call add_task() with a function and its arguments. FastAPI runs it after the response is sent.

Example:

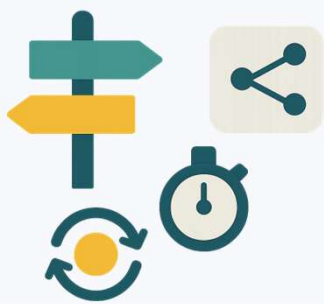
```
from fastapi import FastAPI, BackgroundTasks
import time

app = FastAPI()

# — Task function (regular or async) ————————————————
def send_welcome_email(email: str, username: str) -> None:
        # Simulating slow email sending
        time.sleep(2)
        print(f'[Email] Welcome email sent to {email} for user {username}')


def log_registration(user_id: int, ip: str) -> None:
        print(f'[Log] User {user_id} registered from {ip}')
```



Basic Background Task ->

Routing & Dependencies: Basic Background Task

Example:



```
# Route with background tasks
@app.post('/register', status_code=201)
def register_user(
        username: str,
        email: str,
        background_tasks: BackgroundTasks,
):
        user_id = 42  # normally saved to DB first

        # Schedule tasks — they run AFTER this function returns
        background_tasks.add_task(send_welcome_email, email, username)
        background_tasks.add_task(log_registration, user_id, '127.0.0.1')

        # Response sent immediately — client does NOT wait for email
        return {'user_id': user_id, 'message': 'Registration successful'}
```

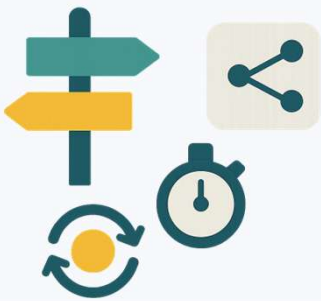
Async Background Task ->

Routing & Dependencies: Async Background Task

Background task functions can be async. FastAPI handles both sync and async tasks transparently. Use async when your task does I/O (HTTP calls, database writes, file operations).

Example:

```python
from fastapi import FastAPI, BackgroundTasks
import asyncio
import httpx
app = FastAPI()

# Async task
async def notify_webhook(url: str, payload: dict) -> None:
async with httpx.AsyncClient() as client:
        try:
                resp = await client.post(url, json=payload, timeout=10.0)
                print(f'[Webhook] POST {url} -> {resp.status_code}')
        except Exception as e:
                print(f'[Webhook] Failed: {e}')

async def update_user_cache(user_id: int) -> None:
        await asyncio.sleep(0.1)  # simulates Redis write
        print(f'[Cache] User {user_id} cache refreshed')
```

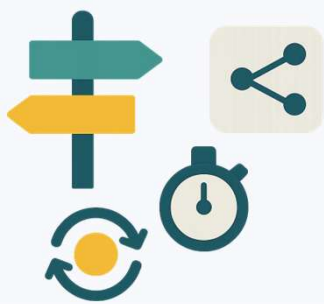


Async Background Task ->

Routing & Dependencies: Async Background Task

Example:

```
@app.put('/users/{user_id}')
async def update_user(
        user_id: int,
        name: str,
        background_tasks: BackgroundTasks,
):
        # Update DB (sync with real SQLAlchemy, or async with asyncpg)
        updated_user = {'id': user_id, 'name': name}

        # Fire-and-forget async tasks
        background_tasks.add_task(update_user_cache, user_id)
        background_tasks.add_task(
                notify_webhook,
                'https://hooks.example.com/user-updated',
                {'user_id': user_id, 'name': name}
        )
        return updated_user   # returned immediately
```



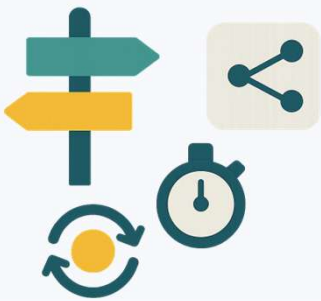
Background Tasks in Dependencies ->

Routing & Dependencies: Background Tasks in Dependencies

You can also inject BackgroundTasks into dependency functions. This lets you encapsulate background task scheduling inside a service layer, keeping route handlers clean.

Example:

```
from fastapi import FastAPI, Depends, BackgroundTasks
app = FastAPI()

# Audit logging service
def write_audit_log(action: str, user_id: int, detail: str):
    print(f'[AUDIT] user={user_id} action={action} detail={detail}')

class AuditService:
    def __init__(self, background_tasks: BackgroundTasks):
        self.bg = background_tasks

    def log(self, action: str, user_id: int, detail: str = ''):
        # Schedules the log write as a background task
        self.bg.add_task(write_audit_log, action, user_id, detail)

def get_audit_service(background_tasks: BackgroundTasks) -> AuditService:
    return AuditService(background_tasks)
```



Background Tasks in Dependencies ->

Routing & Dependencies: Background Tasks in Dependencies

Example:

```
# Routes use AuditService without knowing about BackgroundTasks
@app.delete('/items/{item_id}', status_code=204)
def delete_item(
        item_id: int,
        audit: AuditService = Depends(get_audit_service),
):
        # Delete from DB (simulated)
        audit.log('DELETE_ITEM', user_id=1, detail=f'item_id={item_id}')
        # Audit log written AFTER response — route is clean

@app.post('/items', status_code=201)
def create_item(
        name: str,
        audit: AuditService = Depends(get_audit_service),
):
        item = {'id': 99, 'name': name}
        audit.log('CREATE_ITEM', user_id=1, detail=f'name={name}')
        return item
```

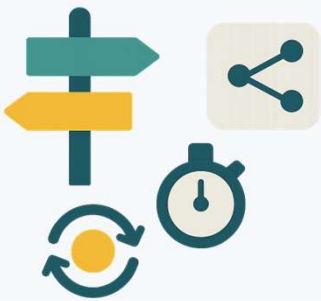
Error Handling in Background Tasks ->

Routing & Dependencies: Error Handling in Background Tasks

Background tasks run AFTER the response is sent. If a background task raises an exception, the client has already received a 200 response — they will NOT see the error. You must handle errors inside the task itself.

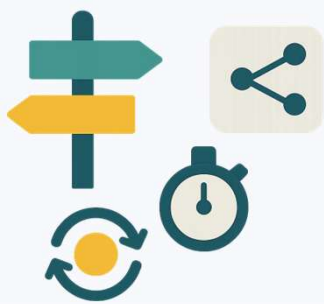
Example:

```python
from fastapi import FastAPI, BackgroundTasks
import logging

app    = FastAPI()
logger = logging.getLogger(__name__)

# Task with proper error handling
def safe_send_email(to: str, subject: str, body: str) -> None:
    try:
        # Simulate potential failure
        if '@' not in to:
            raise ValueError(f'Invalid email address: {to}')
            # real: send via SMTP or API
            print(f'[Email] Sent to {to}: {subject}')
    except Exception as e:
        # CRITICAL: log the error — client won't see it
        logger.error(f'Background email task failed: {e}', exc_info=True)
        # Optionally: write to a dead-letter queue or retry table
```



Error Handling in Background Tasks ->

Routing & Dependencies: Error Handling in Background Tasks



Example: # Task with retry logic

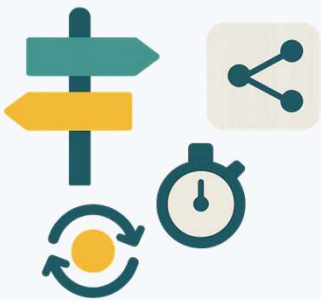
```
import time
def resilient_task(payload: dict, max_retries: int = 3) -> None:
    for attempt in range(1, max_retries + 1):
        try:
            # ... do the work ...
            print(f'[Task] Success on attempt {attempt}')
            return
        except Exception as e:
            logger.warning(f'Attempt {attempt} failed: {e}')
            if attempt < max_retries:
                time.sleep(2 ** attempt)  # exponential back-off
            else:
                logger.error('All retries exhausted', exc_info=True)

@app.post('/notify')
def notify(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(safe_send_email, email, 'Hello', 'World')
    background_tasks.add_task(resilient_task, {'key': 'value'})
    return {'status': 'notification scheduled'}
```

Background Tasks vs External Task Queues ->

Routing & Dependencies: Background Tasks vs External Task Queues

FastAPI's BackgroundTasks runs in the same process. For production workloads, you may need an external task queue. Here is when to use each:

Criterion	BackgroundTasks (FastAPI)	External Queue (Celery/ARQ/etc.)
`Process`	Same as web server	Separate worker processes
`Persistence`	Lost if server crashes	Persisted in Redis/RabbitMQ
`Retries`	Manual with try/catch	Built-in retry policies
`Monitoring`	Logs only	Dashboard + task history
`Concurrency`	Limited by web server threads	Scalable worker pool
`Setup complexity`	Zero — built in	Requires broker + worker
`Best for`	Short, non-critical tasks	Long-running, critical tasks

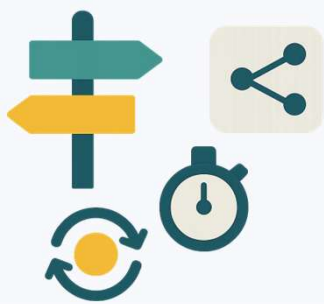


Tip: Use BackgroundTasks for: welcome emails, audit logs, cache invalidation, webhook pings. Use Celery/ARQ for: video encoding, report generation, payment processing, or anything that must survive a server restart.

Combining Background Tasks with Dependencies ->

Routing & Dependencies: Combining Background Tasks with Dependencies

A complete real-world example showing routing, dependency injection, and background tasks working together in a user registration flow:



```python
from fastapi import FastAPI, Depends, BackgroundTasks, HTTPException
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

# — Models ————————————————————————————————————————
class UserCreate(BaseModel):
    username: str
    email: str
    password: str

class UserOut(BaseModel):
    id: int
    username: str
    email: str
```

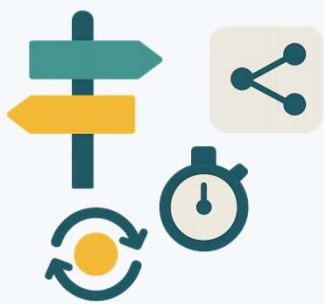
Combining Background Tasks with Dependencies ->

Routing & Dependencies: Combining Background Tasks with Dependencies

```python
# — Dependencies ————————————————————————
fake_users: dict[int, dict] = {}
uid_counter = 0

def get_db():
    return fake_users   # simulated DB


# — Background tasks ————————————————————
def send_welcome_email(email: str, username: str):
    print(f'[BG] Welcome email sent to {email}')

def send_admin_alert(username: str):
    print(f'[BG] Admin: new user {username} registered')

def update_user_count_cache():
    print('[BG] User count cache refreshed')
```

Combining Background Tasks with Dependencies ->

Routing & Dependencies: Combining Background Tasks with Dependencies



```python
# — Route — all three patterns combined ————————————
@app.post('/register', response_model=UserOut, status_code=201)
def register(
    user_data: UserCreate,
    background_tasks: BackgroundTasks,
    db: dict = Depends(get_db),
):
    global uid_counter

    # Check for duplicate
    if any(u['email'] == user_data.email for u in db.values()):
        raise HTTPException(409, 'Email already registered')

    # Save to DB
    uid_counter += 1
    new_user = {'id': uid_counter, 'username': user_data.username,
                'email': user_data.email}
    db[uid_counter] = new_user

    # Schedule background work (client doesn't wait for these)
    background_tasks.add_task(send_welcome_email, user_data.email, user_data.username)
    background_tasks.add_task(send_admin_alert, user_data.username)
    background_tasks.add_task(update_user_count_cache)

    return new_user   # returned immediately
```

Tasks Time ->

Routing & Dependencies : Tasks Time

Task 7: Notification Pipeline with Multiple Background Tasks
Build a user activity notification system. When a user performs certain actions, multiple notifications fire in the background:

- POST /posts — creates a post; background tasks: (1) notify followers via email, (2) update author's post count in cache, (3) index post for search
- POST /orders — places an order; background tasks: (1) send order confirmation email, (2) send warehouse pick list, (3) update inventory counts, (4) charge payment via mock payment API
- DELETE /account — deletes account; background tasks: (1) send goodbye email, (2) archive user data to cold storage, (3) notify admin, (4) free up username

Each background task function must have proper try/except with logging. Add a task_registry list in memory that background tasks append their name to when they complete. Add a GET /task-log endpoint to inspect this registry. Verify all tasks run after the response is returned.

Tasks Time ->

Routing & Dependencies : Tasks Time

Task 8: Audit Logging Service via Background Tasks
Build a complete audit logging system using background tasks and dependency injection:

- An AuditEvent Pydantic model: event_type (str), user_id (int), resource (str), resource_id (Optional[int]), before_state (Optional[dict]), after_state (Optional[dict]), timestamp (datetime), ip_address (str)
- An AuditLogger service class injected via Depends that queues audit events as background tasks
- An in-memory audit store (deque with maxlen=1000) that background tasks write to
- Apply AuditLogger to: POST/PUT/DELETE /products (log create/update/delete with before/after states)
- GET /audit/logs — returns the last N audit events (with pagination), protected by admin dependency
- GET /audit/logs/{user_id} — returns all audit events for a specific user

Write tests using TestClient that verify response is returned before audit log is written, the log entry exists after the response, and audit events have corrected before/after states.

Tasks Time ->

Routing & Dependencies : Tasks Time

Task 9: Resilient Background Task with Retry and Dead-Letter Queue
Build a robust background task system with retry logic and a dead-letter queue (DLQ) for failed tasks:

- A TaskResult dataclass: task_name, args, kwargs, attempt, succeeded, error (Optional[str]), timestamp
- A with_retry(fn, max_retries=3, backoff_base=2) wrapper that retries a task with exponential backoff
- A dead_letter_queue (deque, maxlen=500) where permanently failed tasks are stored
- Three real-looking tasks: send_email (fails if email contains 'fail'), call_webhook (fails if URL contains 'down'), resize_image (always succeeds but is slow)
- POST /upload-image — triggers resize_image in background
- POST /subscribe — triggers send_email in background (test with fail@example.com to trigger DLQ)
- GET /dlq — returns dead letter queue contents (admin protected)
- POST /dlq/{task_id}/retry — manually retries a failed task from the DLQ

Verify the entire lifecycle: task runs, fails, retries with backoff, ends up in DLQ, can be manually retried from the DLQ endpoint.

Background Tasks ->